MicroserviceBase

v. 2.1.0

Nguyen Huynh Tri Cuong

11.05.2026

Contents

Chapter 1

Introduction

1.1 Introduction

MicroserviceBase is a Python + C++ framework for building, discovering, and orchestrating microservices on top of an industry-standard runtime stack: **gRPC** over HTTP/2 for the wire protocol, **HashiCorp Consul** for the service catalog and health checking, and **HashiCorp Nomad** for process orchestration. A single **FastAPI bridge** ties everything together and powers the cross-platform **Manager GUI** (Electron desktop and browser).

It provides:

- **Hexagonal microservice framework**: domain / ports / adapters layout for both Python and C++ services – swap a transport without touching domain code
- **gRPC + reflection**: services advertise methods and message schemas at runtime, no `.proto` file needed at the call site
- **Consul-based service discovery**: services self-register on startup and deregister on clean shutdown; clients resolve via the standard Consul HTTP API
- **Nomad orchestration**: every generated project ships a `deploy/<svc>.nomad.hcl` job (`raw_exec`) with dynamic port allocation; the same file works on Windows and Linux
- **FastAPI bridge**: REST endpoints for scaffold, gRPC, Consul, and Nomad, backing both the Manager GUI and any `curl`-driven script
- **Manager GUI**: dual-host (Electron + browser) dashboard with a Services tab driven by Consul, a Service Network tab for managing the Consul / Nomad agents, and a Service Creator wizard
- **Service Creator wizard & `mb-scaffold` CLI**: emits a complete, ready-to-build service project (Python or C++) with hexagonal layout, Nomad job, Consul registration metadata, optional GUI plugin, and a build/run README
- **Robot Framework integration**: ships a `GrpcClient` connection type for QConnectBase so test cases drive gRPC services with the same `Connect` / `Send Command` / `Verify` / `Disconnect` keywords used for TCP, Serial, SSH, and the legacy RabbitMQ broker

1.1.1 Target Audience

This package is designed for developers and test engineers who need to:

- Build microservices that communicate over gRPC and discover each other via Consul
- Generate complete service projects (Python or C++) from a small specification instead of hand-writing the boilerplate
- Run, monitor, and orchestrate services locally and across machines via Nomad, driven from a graphical dashboard
- Integrate gRPC services into existing Robot Framework test suites without leaving the QConnectBase keyword model
- Deploy a standalone desktop installer (NSIS on Windows, AppImage / `.deb` on Linux) that detects or installs the underlying Consul / Nomad agents

1.1.2 Prerequisites

- Python 3.10 or higher (3.11 recommended)
- HashiCorp **Consul** agent (1.16 or newer) – runs locally as `consul agent -dev` for development; the installer can fetch it via `winget` on Windows or via the official HashiCorp apt / yum repos on Linux
- HashiCorp **Nomad** agent (1.6 or newer) – same install story as Consul
- For C++ services: **vcpkg** (any recent version) and either MSVC (MSYS2 `mingw-w64`) or GCC. Reflection is forced on through a vcpkg `overlay-port`
- For the Manager GUI: optional Node.js + npm if you want to build the Electron shell from source; not required for browser mode

The Python package itself pulls in `grpcio`, `grpcio-tools`, `fastapi`, `uvicorn`, `pydantic`, and `python-consul` as direct dependencies – no message broker is required for the core gRPC + Consul + Nomad path.

The sources of **MicroserviceBase** are available on GitHub.

Installation steps and the desktop installer (*DevAtServGUI*) are documented in the README and in the project's `docs/` folder (C++ runtime install, architecture overview, and changelog).

Chapter 2

Description

2.1 Description

2.1.1 Overview

MicroserviceBase is a framework for building microservices in Python or C++ that talk **gRPC over HTTP/2**, register themselves in a **HashiCorp Consul** service catalog, and run as **HashiCorp Nomad** jobs. A **FastAPI bridge** hosts the REST endpoints used by the cross-platform **Manager GUI**, and the same endpoints are usable from `curl` or any scripting language. All generated code follows **hexagonal architecture** (domain / ports / adapters), so the transport, discovery, and orchestration choices stay confined to the adapter layer and never leak into the business logic.

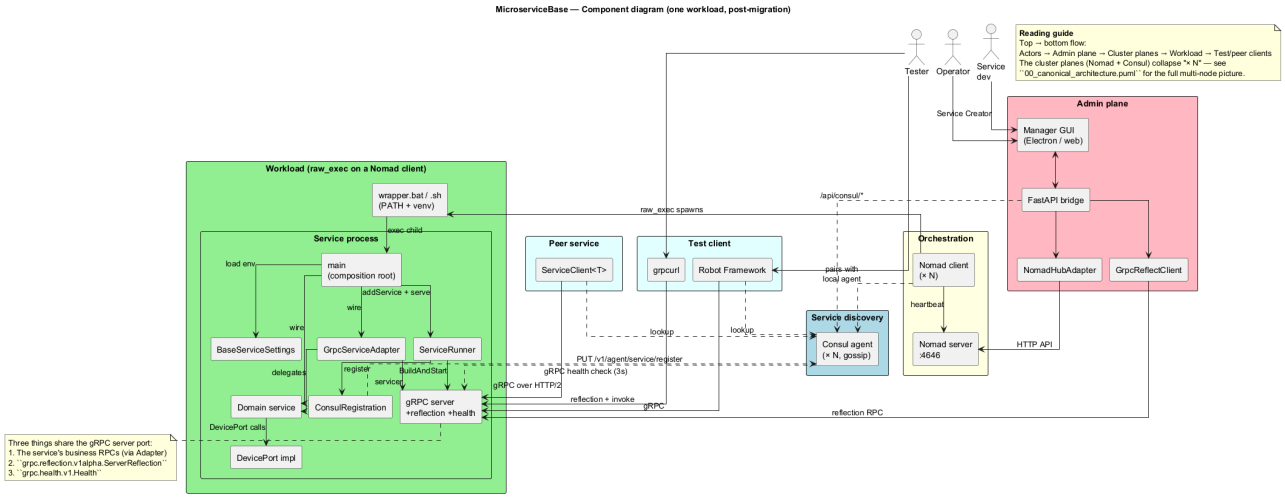


Figure 2.1: MicroserviceBase System Overview

Key Features

- **gRPC + reflection**: services advertise their methods and message schemas at runtime through the standard `grpc.reflection` plugin – clients enumerate without a local `.proto`
- **Consul service catalog**: every service self-registers on startup with name, dynamic port, health check, and a `grpc_services` meta tag, and deregisters cleanly on shutdown
- **Nomad orchestration**: every generated project ships `deploy/<svc>.nomad.hcl` with a single `raw_exec` task and a dynamic `grpc` port; the same job runs on Windows and Linux
- **Hexagonal scaffold for Python and C++**: `mb-scaffold` (CLI) and the **Service Creator wizard** (GUI) emit a complete project with `domain/`, `adapters/`, `proto/`, `deploy/`, build scripts, and an optional `GUIs/` plugin
- **C++ build via vcpkg**: a project-local `vcpkg-overlay-ports/grpc/` forces `gRPC_BUILD_GRPC_CPP_REFLECTION_` so reflection is always available regardless of the upstream port's defaults

- **FastAPI bridge**: `/api/scaffold`, `/api/grpc`, `/api/consul`, `/api/nomad` – one process, four namespaces; used by the GUI and equally by automation scripts
- **Manager GUI**: dual-host (Electron desktop + browser) with a Consul-driven Services sidebar, a Service Network tab for managing the agents, and a Service Creator wizard
- **QConnectBase `GrpcClient`**: contributed upstream so Robot Framework tests drive gRPC services with the same keyword model used for TCP, Serial, SSH, and the legacy RabbitMQ broker
- **Standalone installers**: NSIS for Windows (with Consul / Nomad detection + `winget` install fallbacks), `.deb` / AppImage for Linux (with a `bootstrap.sh` that installs the agents from the official HashiCorp repos)

Design Philosophy

1. **Standards over bespoke**: gRPC, Consul, Nomad – pick the boring, widely-used building blocks instead of inventing transport / discovery / orchestration in-house
2. **Hexagonal architecture**: domain code never imports gRPC, Consul, Nomad, FastAPI, or Qt; swapping a transport is purely an adapter problem
3. **Same code generates same shape**: `mb-scaffold` produces the same file layout for Python and C++, single-service and multi-proto, so a developer familiar with one knows the other on sight
4. **Reflect-first, fall back to compile**: clients prefer gRPC reflection and only compile a local `.proto` (`LocalProtoClient`) when reflection is not available
5. **Graceful shutdown on every platform**: Python services convert `SIGBREAK` (Windows) and `SIGTERM` (Linux) into a clean deregister-then-exit path; C++ services do the same via the `Server` shutdown handler

2.1.2 Architecture

MicroserviceBase is organised into the classic hexagonal layers, applied uniformly to both the framework itself and to every generated service:

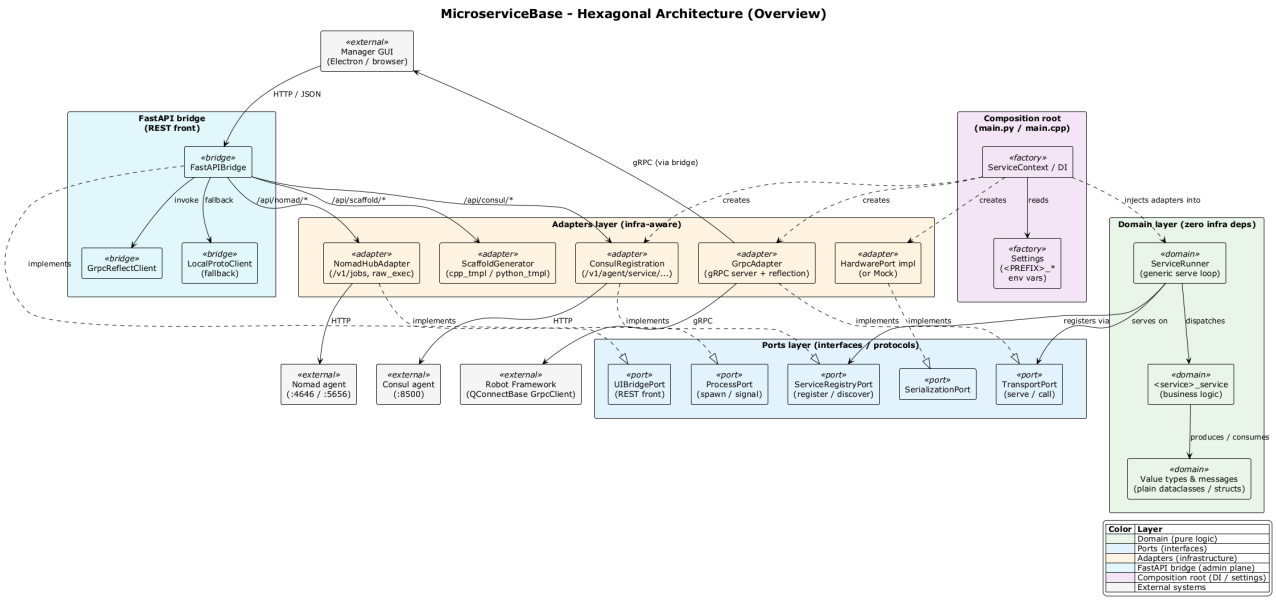


Figure 2.2: Hexagonal Architecture Overview

The detailed architecture with the major adapters (gRPC, Consul, Nomad, FastAPI bridge, scaffold generator) wired through their port interfaces:

Layer Responsibilities

Domain Layer Pure business logic with zero infrastructure dependencies: `ServiceBase` (the abstract microservice contract), value types, and generated service classes (`src/<svc>/domain/<svc>_service.{py,cpp}`). The domain never imports `grpc`, `python-consul`, `fastapi`, `httpx`, or Qt and is unit-testable in isolation.

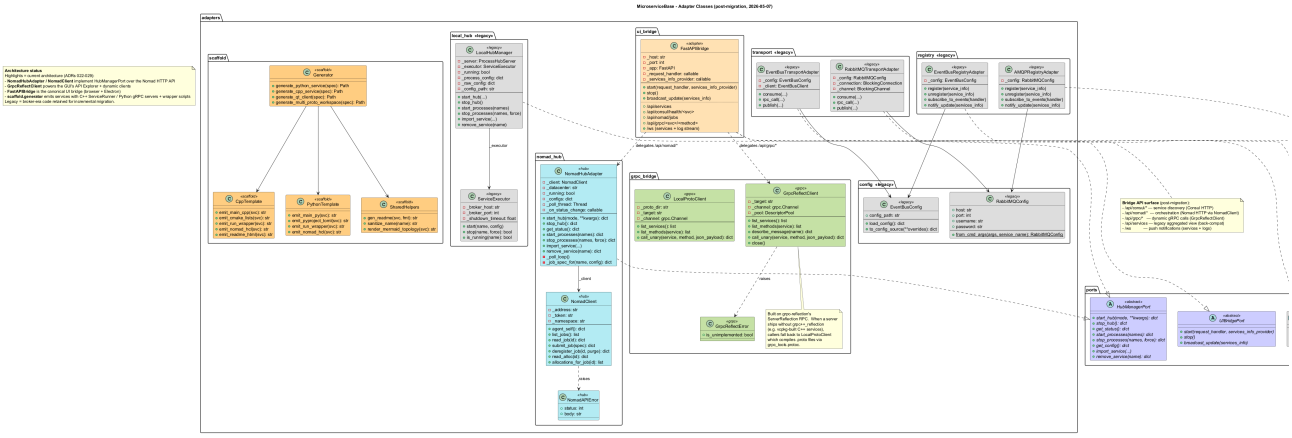


Figure 2.3: Hexagonal Architecture Detail

Ports Layer Abstract protocol interfaces that the domain talks to: `TransportPort` (RPC in / out), `RegistryPort` (registration + discovery), `ProcessPort` (subprocess management), `SerializationPort`, and `UIBridgePort` (HTTP for the GUI).

Adapters Layer Concrete infrastructure-aware implementations:

- `gRPC adapter` – `adapters/grpc_bridge/` – request routing, reflection client (`ReflectClient`), and a `LocalProtoClient` fallback that compiles `.proto` files with `grpc_tools.protoc`
- `Consul adapter` – `adapters/consul/` – HTTP wrapper around `/v1/agent/service/register`, `/v1/health/service/<name>?passing=true`, and the catalog endpoints
- `Nomad adapter` – `adapters/nomad/` – agent lifecycle (start / stop / status), job submission, and per-task status streaming
- `FastAPI bridge` – `adapters/ui_bridge/fastapi_bridge.py` – the single REST front for the GUI; mounts the GUI's static assets in browser mode
- `scaffold generator` – `adapters/scaffold/cpp_tmpl.py` and `python_tmpl.py` – one templated emitter per output file; backs both the wizard and the CLI

GUI Layer `MicroserviceManagerGUI` runs in two modes:

- `Electron` – desktop wrapper with `contextBridge` preload
- `Web` – pure browser-compatible HTML / CSS / JS, served by the FastAPI bridge under `http://localhost:1112`
- `Service Plugins` – per-service HTML + JS dropped into `web/services/` and loaded into the Manager GUI namespace (`window.MicroserviceManager`, alias `MM`)

2.1.3 Components

Service Discovery: Consul

Every service registers itself in Consul on startup with:

- **Name**: snake-case service name (e.g. `analog_input_service`)
- **Address + port**: where its gRPC server is listening (Nomad-allocated, exposed as the `${NOMAD_PORT_grpc}` env var)
- **Health check**: a TCP probe to the gRPC port, polled every few seconds
- **Tags**: `v1` (or whatever the service version is)
- **Meta**: `grpc_services` – comma-separated list of fully-qualified gRPC service names the binary implements (lets the GUI skip a reflection round-trip when listing services to enumerate)

Clients resolve via `GET /v1/health/service/<name>?passing=true` which returns the healthy `address:port` pairs. The Manager GUI uses this for its Services sidebar; `ServiceClient` uses it for outbound gRPC calls. Local-dev typically runs `consul agent -dev` (single-node, in-memory); the Manager GUI's **Service Network → Consul** tab can launch and manage that for you.

Orchestration: Nomad

Each generated service ships a `deploy/<svc>.nomad.hcl` job that declares:

- A single task running the service binary under `raw_exec`
- A dynamic port labelled `grpc` – Nomad picks a free port and exports it as the `${NOMAD_PORT_grpc}` env var
- Environment variables to point the binary at Consul (`CONSUL_ADDR`)

Submit with `nomad job run deploy/<svc>.nomad.hcl`, or via the GUI's Nomad tab → **Submit Job** dialog. `raw_exec` runs the process directly (no container) so the same job works on Windows and Linux without Docker / cgroups setup. For production you can swap to `exec` (Linux container) or `docker` without changing the framework – only the `.hcl` template.

Method Discovery: gRPC Server Reflection

Servers built with our scaffold include `grpc++_reflection` (the vcpkg overlay-port forces it on; see `changelog.html`) and call `grpc::reflection::InitProtoReflectionServerBuilderPlugin()` at startup. Clients can then enumerate services, methods, and message schemas at runtime without any `.proto` file. Fallback: when reflection is not available (older servers, custom builds), the bridge can compile local `.proto` files via `grpc_tools.protoc` and use the descriptors that way – see `LocalProtoClient` in `MicroserviceBase/adapters/grpc_bridge/reflect_client.py`.

The FastAPI Bridge

The bridge is a single Python process that:

1. Hosts the Manager GUI's static assets at `/` (when running in browser mode, default `http://localhost:1112`)
2. Exposes REST endpoints for everything the GUI does:
   - `/api/scaffold/...` – generate service projects (`mb-scaffold` backend)
   - `/api/grpc/...` – list methods + invoke (via reflect_client)
   - `/api/consul/...` – start / stop the agent + read the catalog
   - `/api/nomad/...` – start / stop the agent + submit / stop jobs
3. Manages Consul + Nomad agents as Python subprocesses (PID-tracked in `%TEMP%/msbase_*_agent.pid` for restart resilience)

The GUI is a thin client over this – every button maps to one or two endpoints. The same endpoints are usable from `curl` or scripts when automating. Source: `MicroserviceBase/adapters/ui_bridge/fastapi_bridge.py`.

The Scaffold Generator

`mb-scaffold` (`python -m MicroserviceBase.tools.scaffold_cli`) emits a complete service project from a small spec. Three layout selectors are supported:

`monorepo` (default) – N services share one `.proto`; each service compiles into its own binary

`multi_proto` – N services from N separate `.proto` files, all hosted in one binary on one port with one Consul registration

`separate` – one service per project (degenerate case of monorepo)

For Python the scaffold emits `main.py`, `config.py`, `context.py`, `pyproject.toml`, the hexagonal `domain/` and `adapters/api/` folders, the `proto/` contract plus `scripts/generate_protos.py`, and optional `deploy/` (Nomad job + `*_service_config.json` for Consul) and `GUIs/` (HTML + JS plugin) directories. For C++ it additionally emits `CMakeLists.txt`, `vcpkg.json`, the `vcpkg-overlay-ports/grpc/` folder, and `build_msvc.bat` / `build_msys2.bat` wrappers. Source: `MicroserviceBase/adapters/scaffold/` (cpp_tmpl.py + python_tmpl.py).

Generated Service Layout

The same shape is produced for every service. Files marked *generic* are templated and safe to regenerate; files marked *specific* hold the parts you actually write (your `.proto` contract and the corresponding domain class):

```
out/<service>/
+-- main.py | main.cpp           # Entry point (generic)
+-- config.py | config.h         # Settings, env vars, ports        (generic)
+-- context.py                   # ServiceContext / DI              (generic, Python)
+-- pyproject.toml               # uv / pip deps                    (generic, Python)
+-- CMakeLists.txt | vcpkg.json  # Build files                      (generic, C++)
+-- vcpkg-overlay-ports/grpc/    # Forces reflection plugin ON      (generic, C++)
+-- proto/<svc>.proto            # gRPC contract                    (SPECIFIC)
+-- domain/<svc>_service.{py,cpp}  # Business logic                 (SPECIFIC)
+-- adapters/api/grpc_adapter.{py,cpp}  # Servicer (proto -> domain) (generic)
+-- scripts/generate_protos.py   # Pre-builds *_pb2[_grpc].py       (generic, Python)
+-- deploy/<svc>.nomad.hcl       # Nomad raw_exec job               (generic)
+-- deploy/<svc>_service_config.json  # Consul registration meta    (generic)
+-- GUIs/<svc>.html | <svc>.js   # Optional Manager GUI plugin      (generic + custom)
+-- README.md                    # Build / run / deploy steps       (generic)
```

Re-running `mb-scaffold` on an existing project overwrites the generic files and leaves the specific ones alone, so your `.proto` and domain code survive a regen.

2.1.4 Communication Patterns

RPC via gRPC over HTTP/2

Clients call services through generated stubs (when they have the `.proto`) or via the bridge's reflection-driven generic invoker (when they do not). Below is the generic-invoker path in Python – the same call from JavaScript or Robot Framework goes through the bridge in exactly the same way:

```
import httpx

# Resolve via Consul (handled by the bridge)
r = httpx.post("http://localhost:1112/api/grpc/invoke", json={
    "service": "calculator_service",            # Consul service name
    "method":  "Calculator.Add",                # FQN: <package>.<svc>.<method>
    "request": {"a": 1, "b": 2},                # message as JSON
})
print(r.json())   # {"result": 3}
```

The bridge looks the service up in Consul, opens a gRPC channel, calls the method (synthesising stubs from reflection if required), and returns the response as JSON. Stub-based calls for performance-critical paths are documented in the per-language `README` that ships with every generated project.

Service Registration

Registration is automatic. In Python, `ServiceBase.register_service()` is called from the generated `main.py` after the gRPC server has bound to its port; in C++, the equivalent call lives in `main.cpp` after `Server::BuildAndStart()`. Both speak directly to the local Consul agent over HTTP. On clean shutdown the service deregisters itself; if the process dies abruptly, Consul's TCP health check fails it out of the catalog within a few seconds.

Robot Framework: `GrpcClient`

The QConnectBase `GrpcClient` connection type ships with this release (`QConnectBase/grpc/grpc_client.py`). Robot tests drive gRPC services with the same `Connect` / `Send Command` / `Verify` / `Disconnect` keywords already in use for TCP, Serial, SSH, and the legacy RabbitMQ broker – and support both modes:

- **Direct** – supply `host:port` explicitly

- **Consul-resolved** – supply `service_name` + `consul_addr` and let the client resolve the endpoint at `Connect` time

The client uses gRPC reflection by default and falls back to `LocalProtoClient` (local `.proto` compilation) when reflection is unavailable.

2.1.5 GUI Architecture

Dual Hosting

The MicroserviceManagerGUI runs in two modes:

Electron Mode Desktop application with native process management. Uses `contextBridge` preload for secure Node.js access. Can spawn and manage the FastAPI bridge process (and, indirectly, the Consul / Nomad agents) directly.

Browser Mode Access via `http://localhost:1112` when the bridge is running. Pure HTML / CSS / JS with no Node.js dependencies. Uses the same `window.MicroserviceManager` (alias `MM`) namespace as Electron mode.

Service Plugins

Each microservice can ship a custom GUI plugin (when `--with-gui` is passed to `mb-scaffold`, or the wizard's *Generate GUI template* toggle is enabled):

- One HTML file (Bootstrap 5 card layout)
- One JS file (`const MM = window.MicroserviceManager;`); exposes `load<ServiceName>()` and `unload<ServiceName>()`
- Plugins are loaded dynamically into `web/services/` at runtime
- In packaged mode (DevAtServGUI), plugins are extracted into the writable `%APPDATA%/devatservgui/` directory on first run

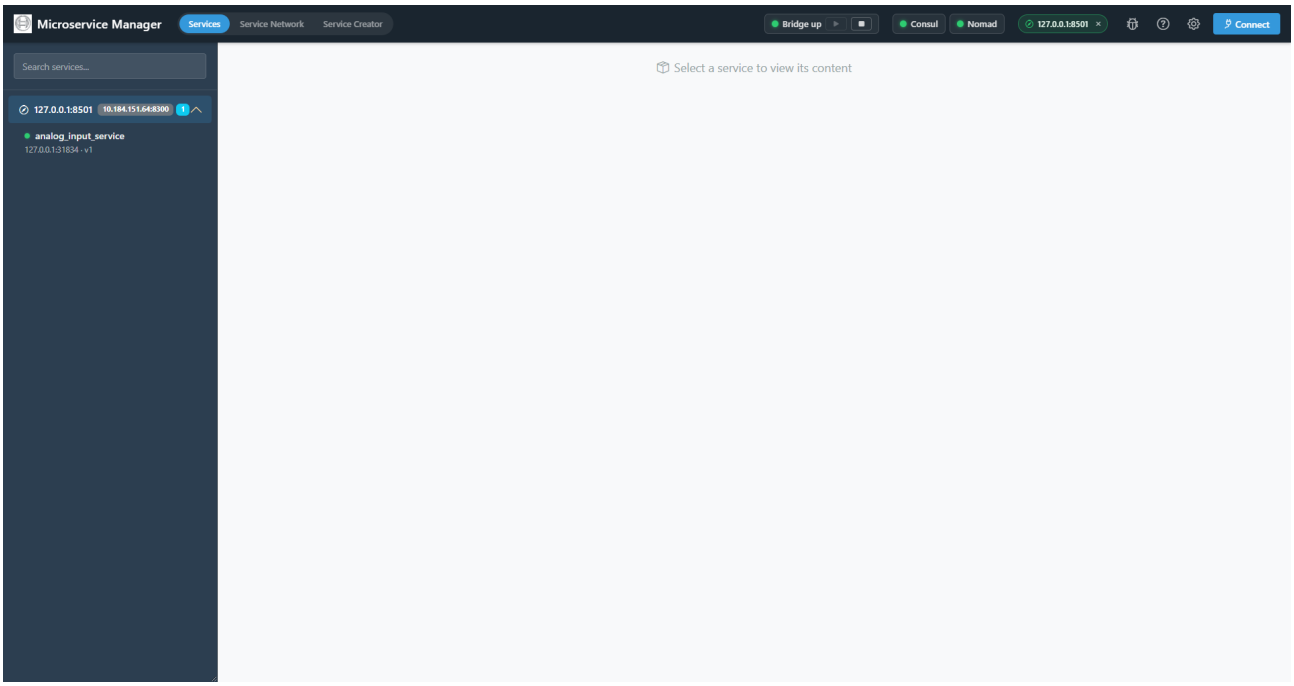
Standalone Installer

The GUI can be packaged as a standalone application:

- **Windows** – NSIS installer (`build.bat`). The *Infrastructure* wizard page detects Consul / Nomad on `PATH`, in `%ProgramFiles%\HashiCorp\`, in the WinGet `Links` shim folder, and in `WinGet\Packages\Hashicorp.*`; offers `winget install` (pinned), *Browse existing path*, or *Skip*.
- **Linux** – `.deb` + AppImage (`build.sh`); ships `bootstrap.sh` in `extraResources` that detects via `command -v` and installs from HashiCorp's official apt / yum repos with version pinning, falling back to release-tarball download.
- Read-only scripts in `resources/python/`, writable user data in `%APPDATA%/devatservgui/` (Windows) or `$XDG_CONFIG_HOME/devatservgui/` (Linux).
- Bridge process is managed via PID file with detached mode so it survives the GUI being closed.

2.1.6 GUI User Guide

The Manager GUI provides three top-level tabs: **Services** for browsing and calling registered microservices, **Service Network** for managing the underlying Consul and Nomad agents, and **Service Creator** for scaffolding new microservices. Two infrastructure pills in the top-right corner show whether the local Consul and Nomad agents are running; clicking either pill jumps straight to the matching tab.



Manager GUI – top tabs, infrastructure pills, and Consul connection chips

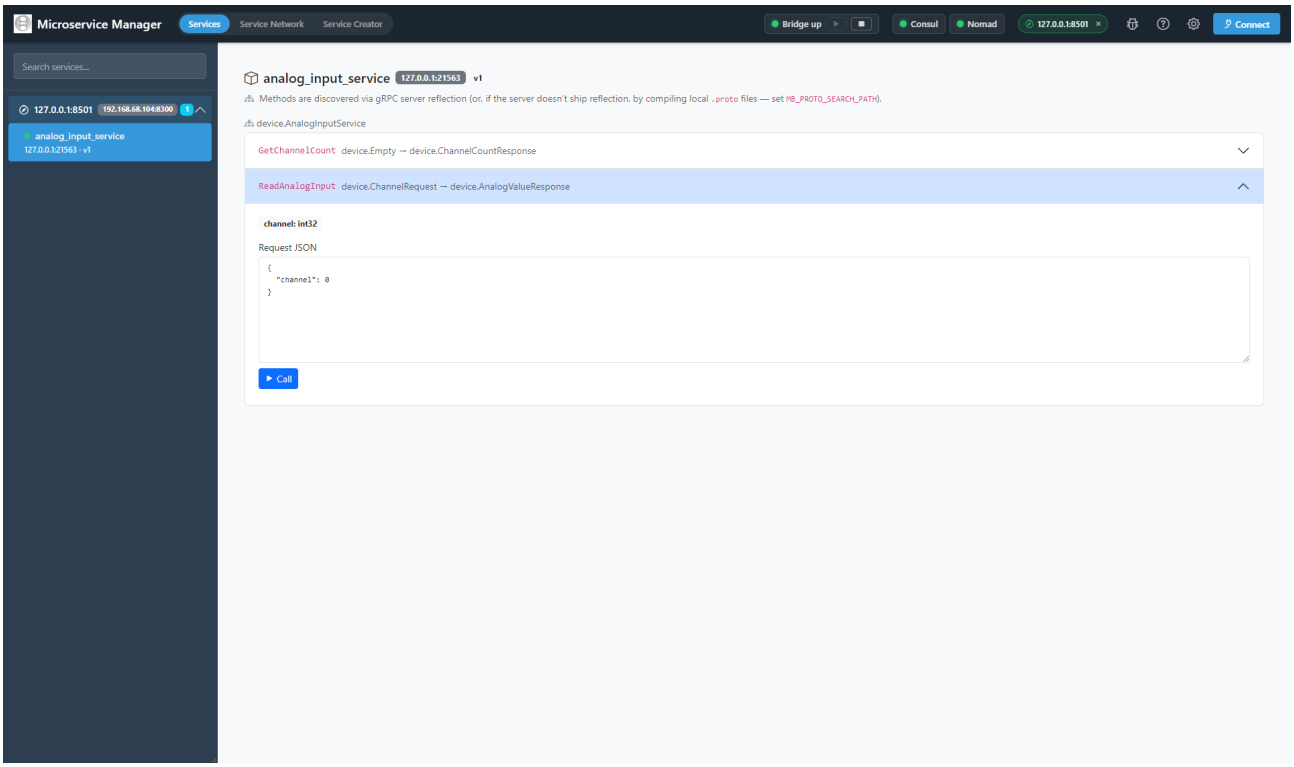
Connecting to Consul

Click **Connect** in the top-right corner to attach the GUI to a Consul cluster. The Connect modal asks for a **Consul URL** (default `http://localhost:8500`); a **Detect** button scans local processes for a running `consul agent` and pre-fills the URL. Multiple Consul URLs can be connected at once – each appears as a chip in the top-right corner with an `x` button to disconnect.

Services Tab

Once at least one Consul URL is connected, the **Services** sidebar lists every registered service grouped by Consul (when more than one cluster is connected) and then by service `group` meta. Each service row is an accordion; expanding it shows the methods discovered through gRPC reflection. Selecting a service:

- Loads its custom GUI plugin into the main panel (when `gui_support` is set), or shows a generic method-list view
- Opens a per-method *Helper* dialog with copy-paste-ready Python, JavaScript, and Robot Framework snippets
- Provides a generic *Invoke* form populated from the message schema, for quick smoke testing without writing code



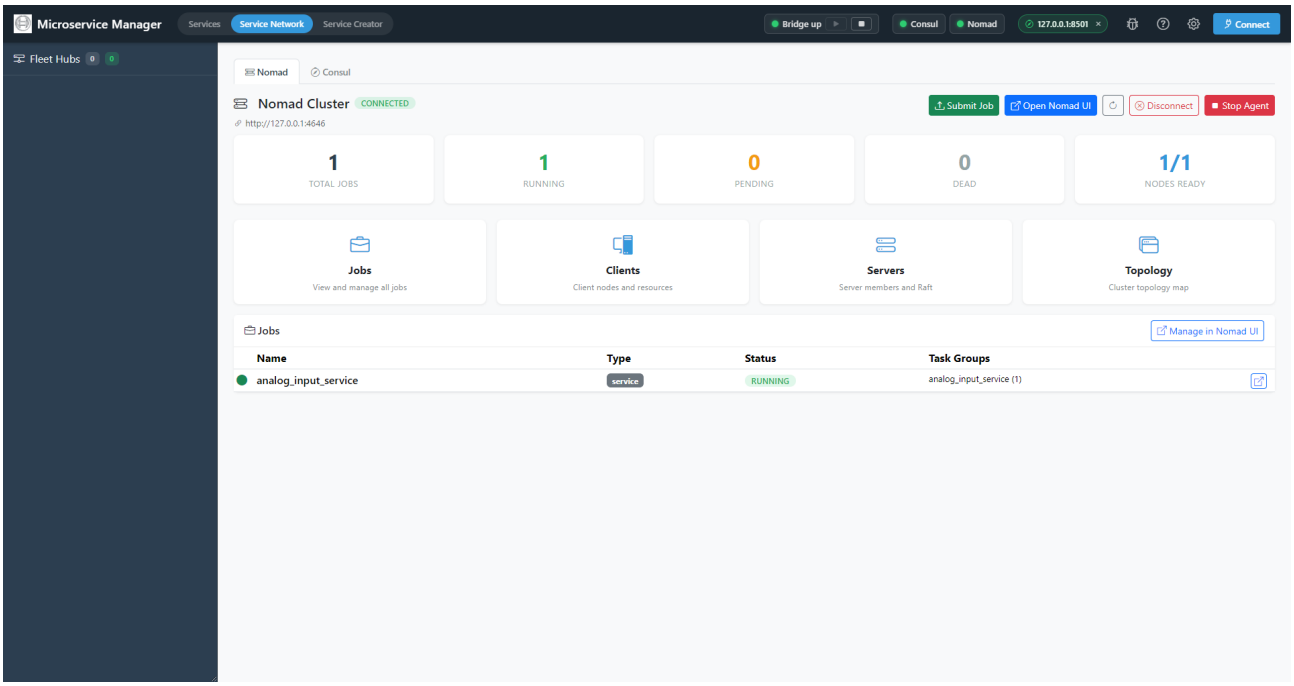
Services tab – API explorer with reflection-driven method list and generic invoke form

Service Network Tab

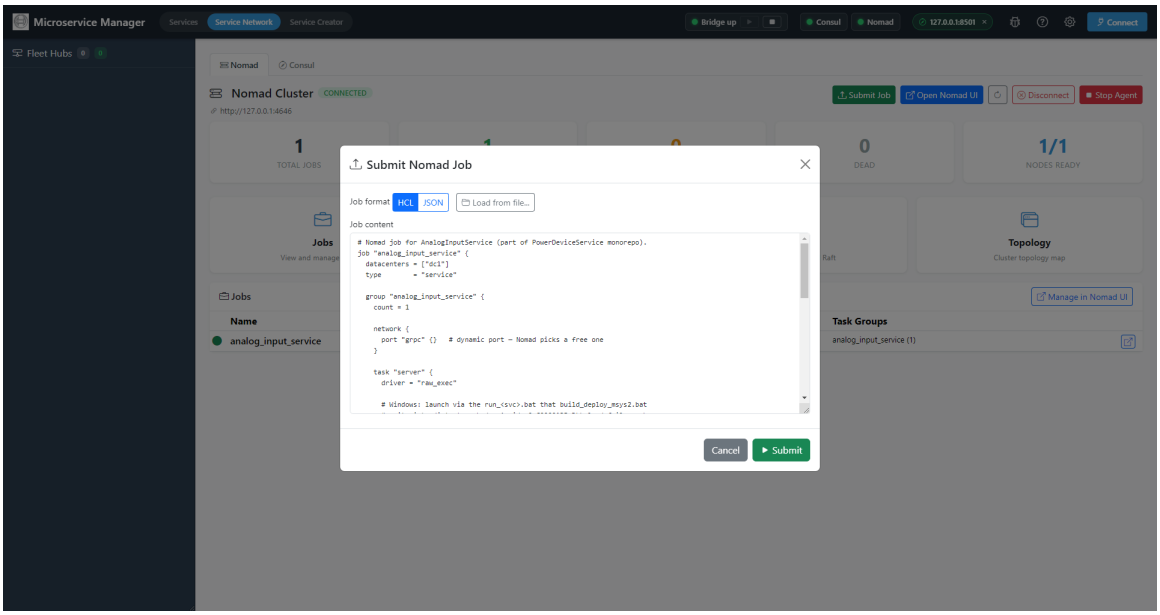
The **Service Network** tab has two sub-tabs: **Nomad** (default) and **Consul**. Both follow the same pattern: a *status* card at the top with **Start agent** / **Stop agent** buttons (configurable address, data dir, log file), and a *dashboard* below that lists what the agent currently knows about.

Nomad sub-tab Shows running jobs, allocations, and per-task status. A **Submit Job** dialog accepts a `.nomad.hcl` file picker (or paste-text), parses it locally, and POSTs to `/v1/jobs`. Job logs stream into a side panel via the Nomad agent's `logs` endpoint.

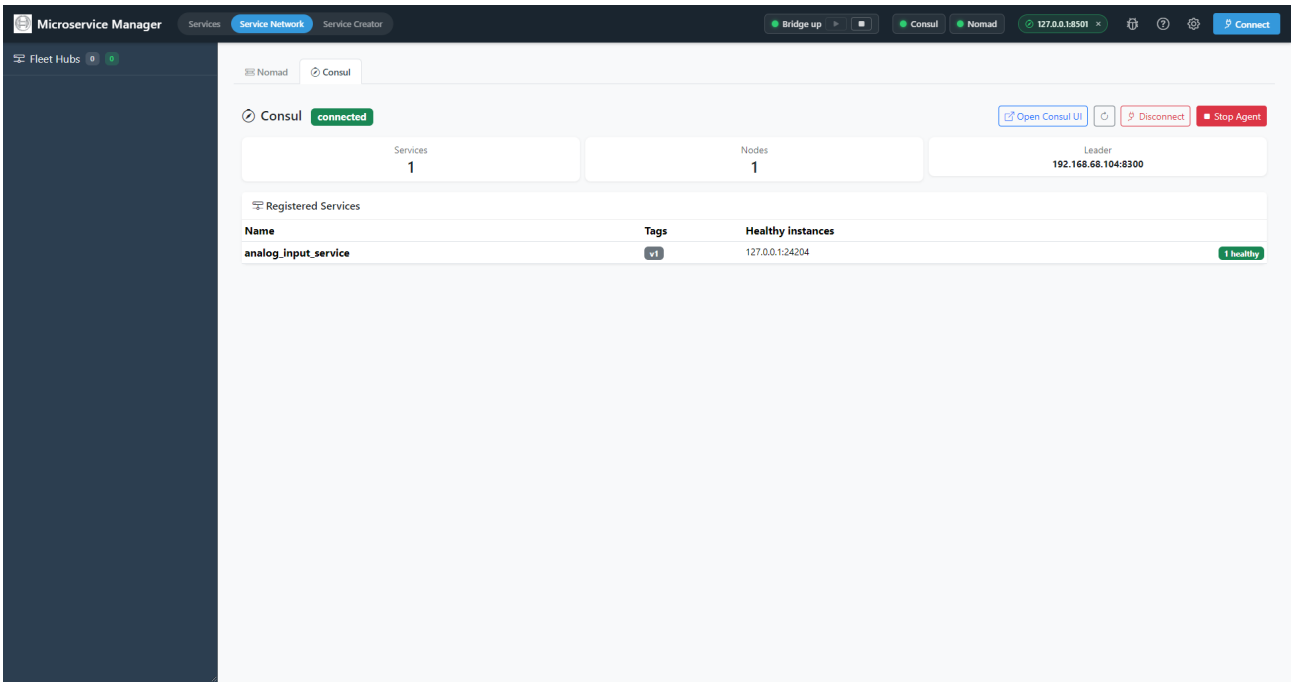
Consul sub-tab Shows the service catalog as Consul sees it – service names, IDs, host:port pairs, tags, health-check status, and the `grpc_services` meta. Useful for confirming a freshly-deployed service really did register (and that its health check is passing).



Service Network – Nomad dashboard with jobs, allocations, and per-task status



Submit Job dialog – file picker for `.nomad.hcl`

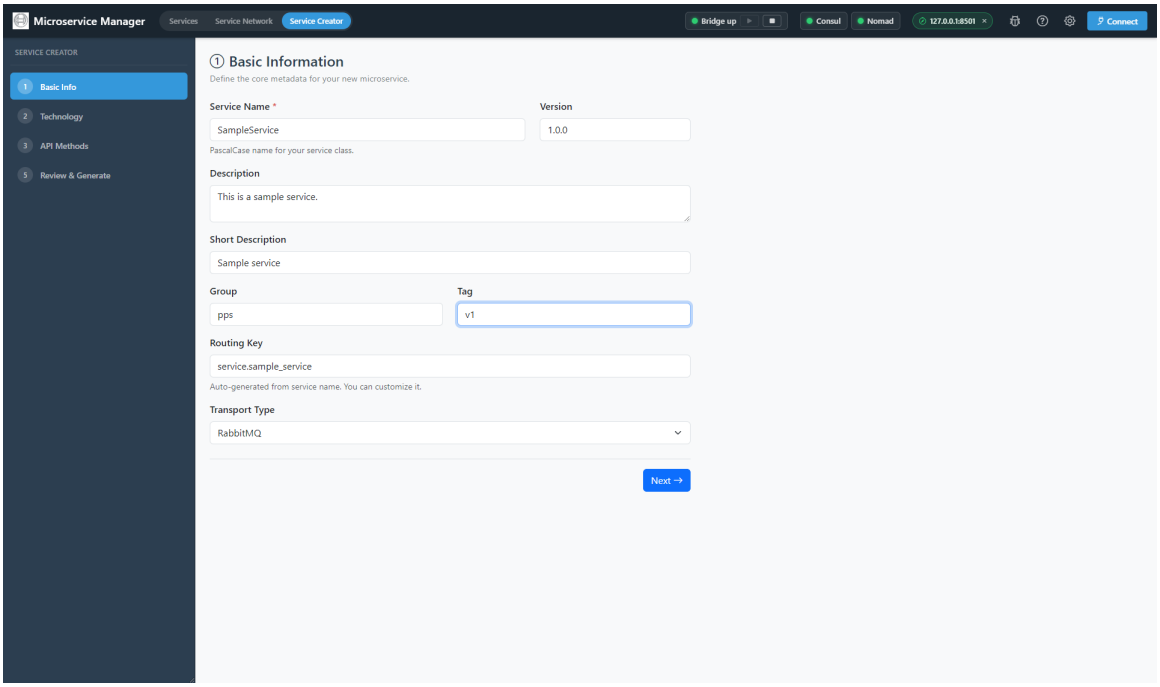


Service Network – Consul service catalog with health-check status

Service Creator Wizard

The **Service Creator** tab is a four-step wizard for generating a new service project. Behind the scenes it POSTs to `/api/scaffold/generate-v2`; the same specification can be given to `mb-scaffold` on the command line.

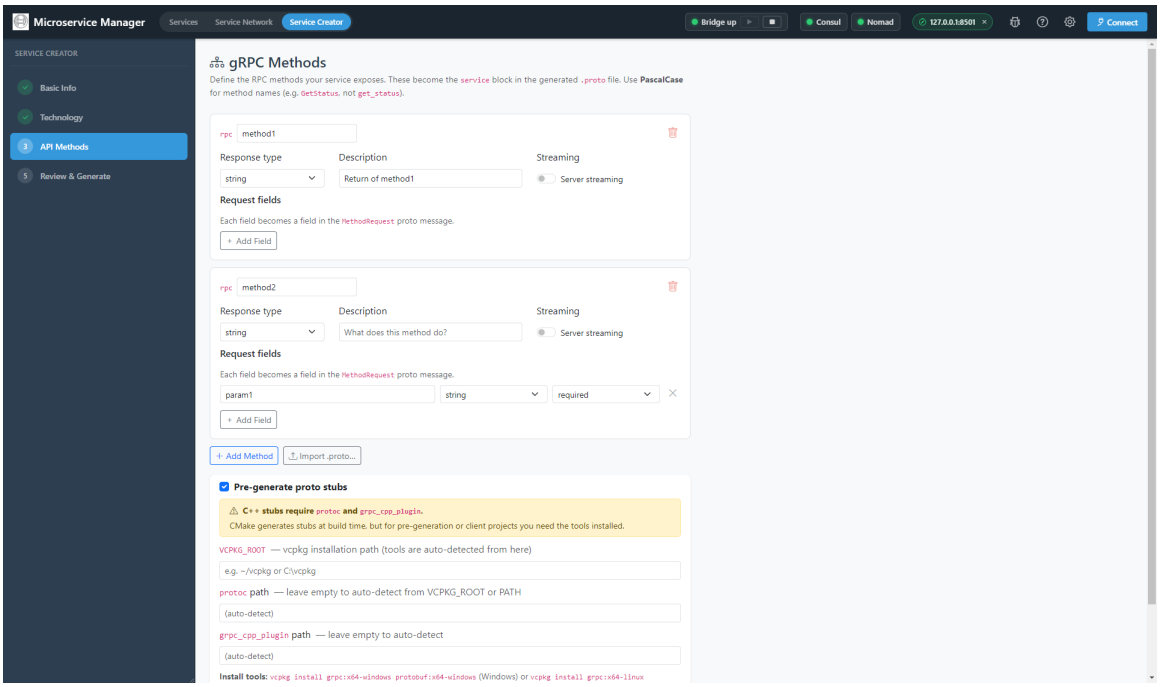
Step 1: Basic Information



Service Creator – Step 1: Basic Information

Define the core metadata for the new service: **Service Name** (PascalCase), **Version**, **Description / Short description**, **Group** (for sidebar organisation in the GUI), **Tag**, and **Language** (Python or C++). The **Layout** selector picks between `monorepo`, `multi_proto`, and `separate`. In `multi_proto` mode the wizard also lets you import multiple existing `.proto` files in one shot.

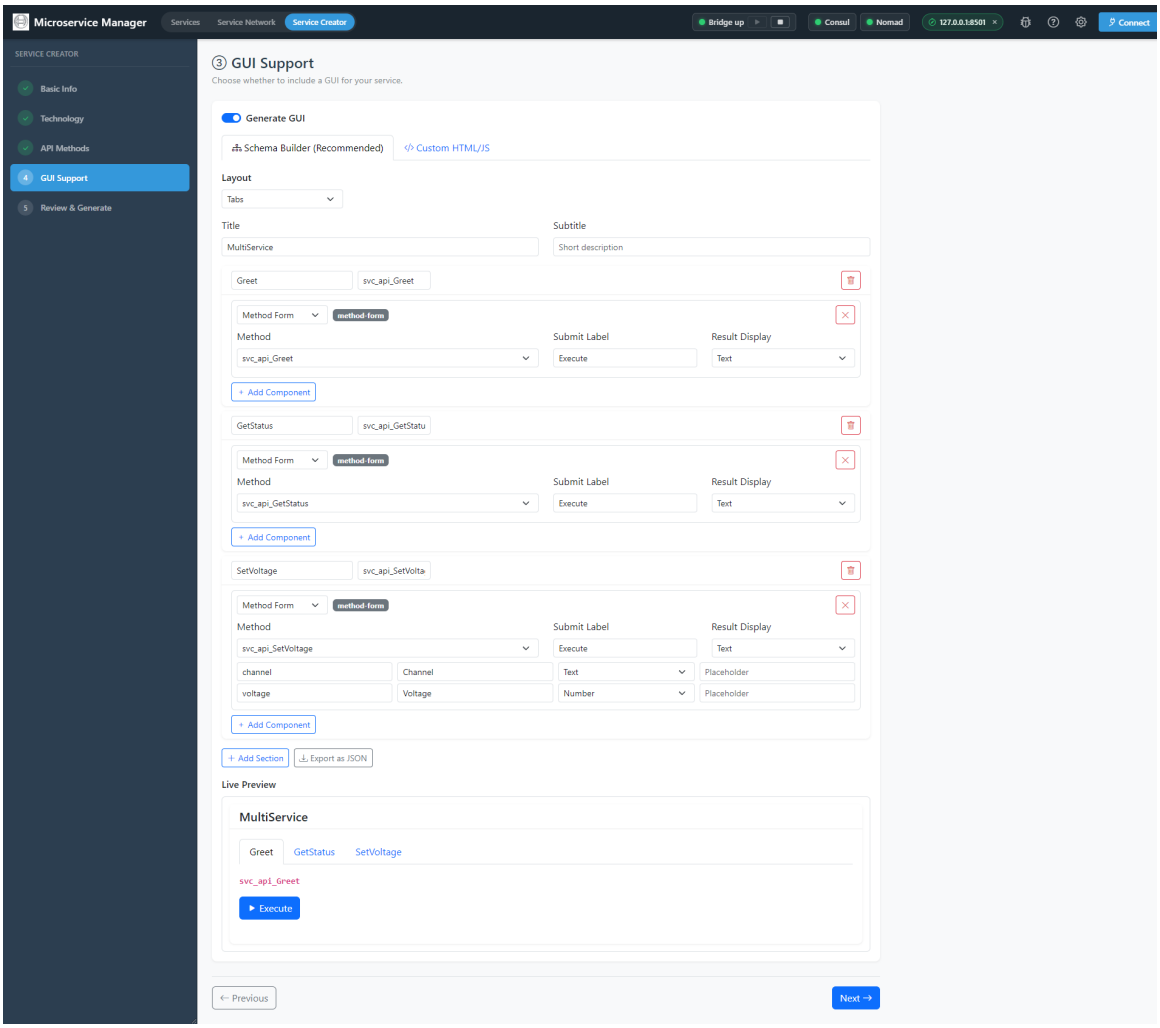
Step 2: API Methods



Service Creator – Step 2: API Methods

Define the methods the service will expose. Each method has a name (becomes a gRPC RPC), a return type, a description (becomes the docstring / Doxygen block), and a list of parameters with name + type + condition (`required` / `optional`). The wizard renders the equivalent `.proto` text live in a side panel.

Step 3: GUI Support

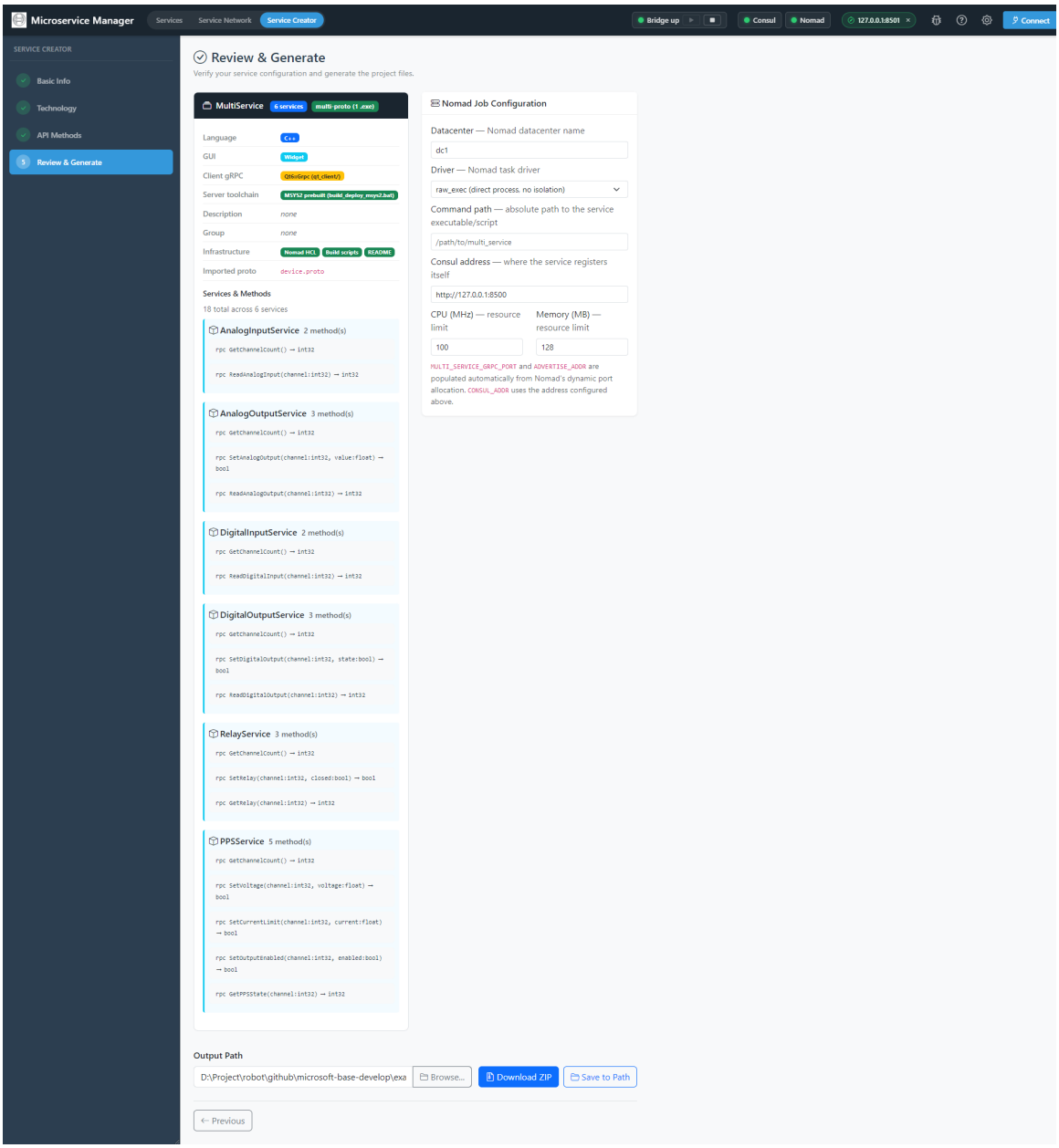


Service Creator – Step 3: GUI Support

Choose whether to include a custom GUI plugin. When the *Generate GUI template* toggle is on, the wizard offers two complementary editors that you can mix and match:

- **Schema Builder** – a visual form designer that emits a Bootstrap card + JS pair driven by a JSON schema (best for parameter-form services)
- **Custom HTML / JS editor** – a Monaco-style live editor with a *Live preview* panel (best for non-form layouts; full access to `window.MicroserviceManager` / `MM`)

Step 4: Review and Generate



Service Creator – Step 4: Review and Generate

The summary card shows every choice you made – name, layout, language, methods, GUI toggle. The **Generated File Structure** preview lists every file the wizard is about to write, with the same generic / specific colouring used elsewhere in the docs. Specify an **Output Path** to save directly to disk, or leave it blank to **Download ZIP**. Once generated, the project can be built and submitted to Nomad with the commands printed in the project's own `README.md`.

2.1.7 Nomad Job Configuration

Each generated service gets a `deploy/<svc>.nomad.hcl` job – a short HCL file that Nomad runs verbatim. A typical multi-port service looks like:

```
job "calculator_service" {
  datacenters = ["dc1"]
  type        = "service"

  group "calc" {
    count = 1

    network {
      port "grpc" {}                # dynamic port
```

```
    }

    service {
      name = "calculator_service"
      port = "grpc"
      tags = ["v1"]
      meta {
        grpc_services = "calc.Calculator"
      }
      check {
        type     = "tcp"
        port     = "grpc"
        interval = "5s"
        timeout  = "1s"
      }
    }

    task "calc" {
      driver = "raw_exec"
      config {
        command = "${NOMAD_TASK_DIR}/calculator_service.exe"
        args    = []
      }
      env {
        CONSUL_ADDR = "http://127.0.0.1:8500"
        GRPC_PORT   = "${NOMAD_PORT_grpc}"
      }
      resources { cpu = 100 memory = 128 }
    }
  }
}
```

The Service Creator wizard generates this file for every service automatically. For multi-proto layouts, a single job hosts all the services on one port and registers each one separately under its own Consul name.

2.1.8 Process Lifecycle

The lifecycle of a service process is the same on Windows and Linux:

1. **Spawn** – Nomad's `raw_exec` driver launches the binary with a dynamic `NOMAD_PORT_grpc` and the `CONSUL_ADDR` env var
2. **Bind & Register** – the service binds the gRPC server, then calls `register_service()` against the local Consul agent
3. **Serve** – Consul's TCP health check polls the gRPC port; the bridge resolves the service via Consul and routes calls
4. **Shutdown** – on `SIGBREAK` (Windows) or `SIGTERM` (Linux), the service deregisters from Consul and lets the gRPC server drain. Critical Windows fix: the default `SIGBREAK` handler calls `ExitProcess` (no Python `KeyboardInterrupt`, no `finally` blocks) – the framework explicitly installs `signal.default_int_handler` so SIGBREAK becomes `KeyboardInterrupt` and the deregister-then-exit path runs cleanly

2.1.9 Installation

From Source

```
git clone https://github.com/test-fullautomation/python-microservice-base.git
cd python-microservice-base
pip install .

# Development mode
pip install -e .
```

The Consul and Nomad agent binaries must be installed separately. On Windows: `winget install Hashicorp.Consul Hashicorp.Nomad`. On Debian / Ubuntu: follow the official `apt` repo instructions at HashiCorp's docs. The `bootstrap.sh` script in the Linux installer does this non-interactively for you.

Standalone Desktop Installer (DevAtServGUI)

- Run `build.bat` (Windows, NSIS) or `build.sh` (Linux) inside `MicroserviceBase/MicroserviceManagerGUI/`
- The Windows installer detects existing Consul / Nomad and offers `winget install` when missing
- The Linux installer ships `bootstrap.sh` for AppImage users; `.deb` users get a `postinst` hook that runs it when stdin is a TTY

GUI from Source

```
cd MicroserviceBase/MicroserviceManagerGUI
npm install
npm start            # Electron mode
# or, for browser mode, just start the bridge and open http://localhost:1112
```

2.1.10 Quick Start

Start the Local Agents

```
consul agent -dev                                        # one terminal
nomad agent -dev -bind=0.0.0.0 -network-interface=lo     # another terminal
```

(Or use the Manager GUI's **Service Network** tab to start both as managed subprocesses.)

Start the FastAPI Bridge

```
python -m MicroserviceBase.adapters.ui_bridge.fastapi_bridge \
    --bridge-port 1112 --consul-addr http://127.0.0.1:8500
```

Open `http://localhost:1112` for the Manager GUI in browser mode.

Generate and Run a Service

```
# Python single-service:
python -m MicroserviceBase.tools.scaffold_cli \
    --name calculator_service --language python \
    --layout monorepo --out out/calculator_service \
    --method "Add(a:int,b:int)->int" \
    --method "Multiply(a:int,b:int)->int"

cd out/calculator_service
pip install -r requirements.txt
python scripts/generate_protos.py            # builds *_pb2[_grpc].py
nomad job run deploy/calculator_service.nomad.hcl
```

The service registers itself in Consul on startup; the Manager GUI's Services tab picks it up automatically.

Call the Service from Robot Framework

```
*** Settings ***
Library    QConnectBase

*** Test Cases ***
Calculator Should Add
    Connect       grpc://calculator_service@http://127.0.0.1:8500
```

```
${result}=    Send Command    Calculator.Add    {"a": 1, "b": 2}
Verify        ${result}       {"result": 3}
Disconnect
```

The `grpc://` URL form selects the QConnectBase `GrpcClient` connection type; the `<service_name>@<consul_addr>` suffix tells it to resolve via Consul before opening the channel.

2.1.11 Diagrams Reference

The following PlantUML diagrams ship in the `docs/diagrams/` folder. Older diagrams from the RabbitMQ era are preserved under `docs/diagrams/_archive/` for historical reference; the active set below reflects the current gRPC + Consul + Nomad architecture:

00_canonical_architecture.puml Canonical end-to-end picture: services, Consul, Nomad, FastAPI bridge, GUI, Robot Framework client

component.puml Component diagram with package dependencies (current layout)

class_domain.puml Class diagram for the domain layer (`ServiceBase`, value types, generated service classes)

class_ports.puml Class diagram for port interfaces (Transport, Registry, Process, Serialization, UIBridge)

class_adapters.puml Class diagram for adapter implementations (gRPC, Consul, Nomad, FastAPI bridge, scaffold)

gui_architecture.puml GUI dual-host architecture (Electron + browser)

sequence_communication.puml End-to-end RPC: GUI – bridge – Consul – service

sequence_rpc.puml Single-RPC sequence (reflection-driven invocation)

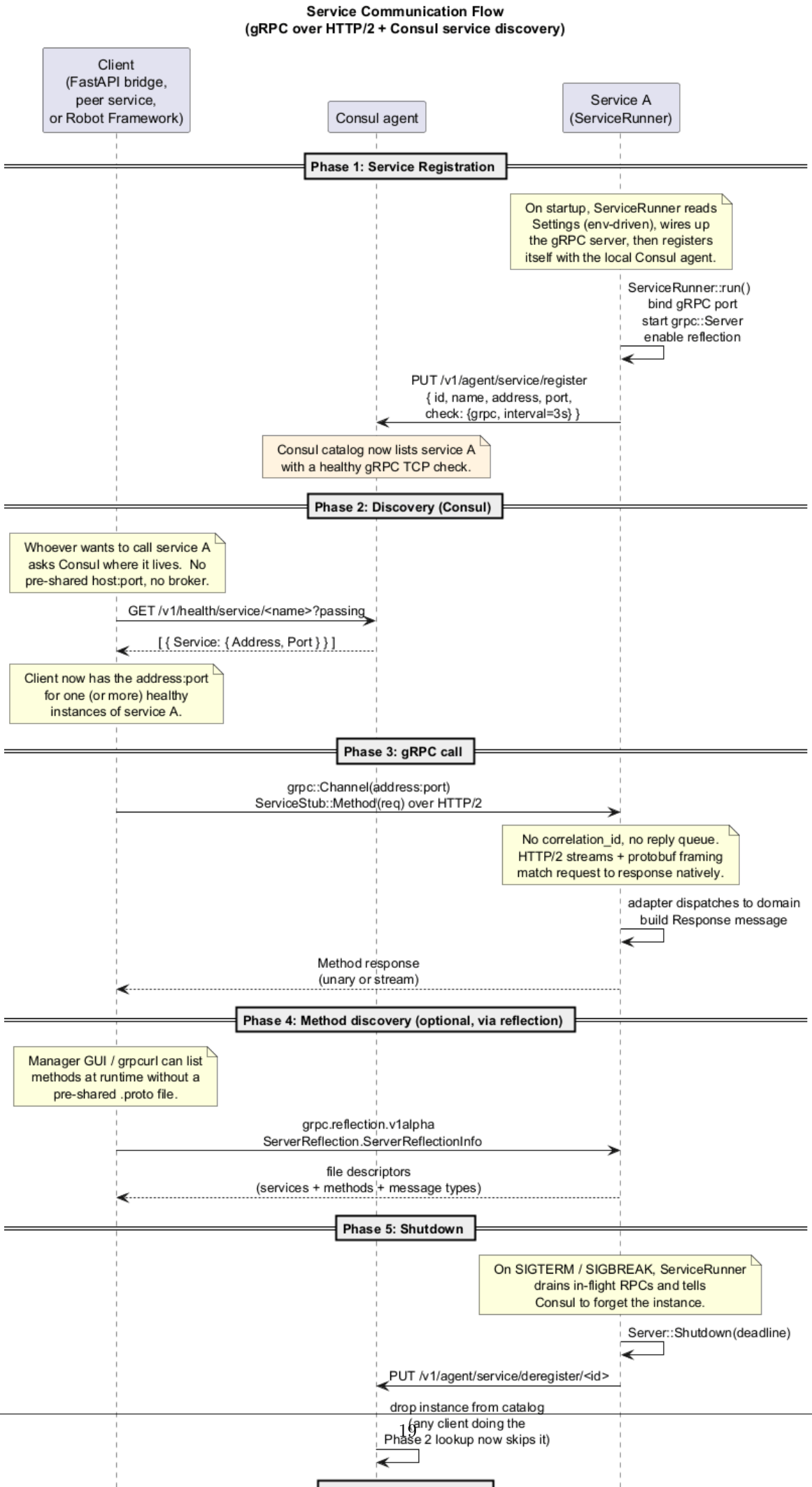
sequence_registration.puml Service registration flow (Consul HTTP)

sequence_shutdown.puml Graceful deregister-then-shutdown sequence

sequence_gui_plugin_loading.puml How service plugins are loaded into the Manager GUI namespace

state_process_lifecycle.puml Service process state machine (spawn, register, serve, deregister, exit)

component_qt_*.puml / sequence_qml_*.puml Qt-shell template family (Widget, QML, WASM) used by the Service Plugin host


Service Communication Flow
(gRPC over HTTP/2 + Consul service discovery)
Client
(FastAPI bridge,
peer service,
or Robot Framework)
Consul agent
Service A
(ServiceRunner)
Phase 1: Service Registration
On startup, ServiceRunner reads
Settings (env-driven), wires up
the gRPC server, then registers
itself with the local Consul agent.
ServiceRunner::run()
bind gRPC port
start grpc::Server
enable reflection
PUT /v1/agent/service/register
{ id, name, address, port,
check: {grpc, interval=3s} }
Consul catalog now lists service A
with a healthy gRPC TCP check.
Phase 2: Discovery (Consul)
Whoever wants to call service A
asks Consul where it lives. No
pre-shared host:port, no broker.
GET /v1/health/service/<name>?passing
[{ Service: { Address, Port } }]
Client now has the address:port
for one (or more) healthy
instances of service A.
Phase 3: gRPC call
grpc::Channel(address:port)
ServiceStub::Method(req) over HTTP/2
No correlation_id, no reply queue.
HTTP/2 streams + protobuf framing
match request to response natively.
adapter dispatches to domain
build Response message
Method response
(unary or stream)
Phase 4: Method discovery (optional, via reflection)
Manager GUI / grpcurl can list
methods at runtime without a
pre-shared .proto file.
grpc.reflection.v1alpha
ServerReflection.ServerReflectionInfo
file descriptors
(services + methods + message types)
Phase 5: Shutdown
On SIGTERM / SIGBREAK, ServiceRunner
drains in-flight RPCs and tells
Consul to forget the instance.
Server::Shutdown(deadline)
PUT /v1/agent/service/deregister/<id>
drop instance from catalog
(any client doing the
Phase 2 lookup now skips it)

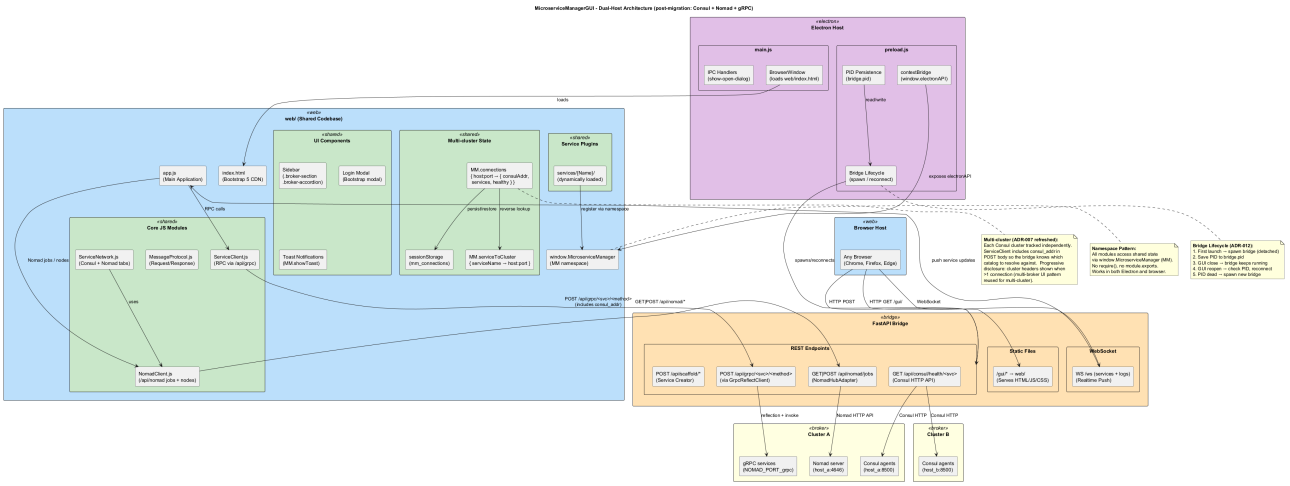


Figure 2.5: GUI Dual-Host Architecture (Electron + Browser)

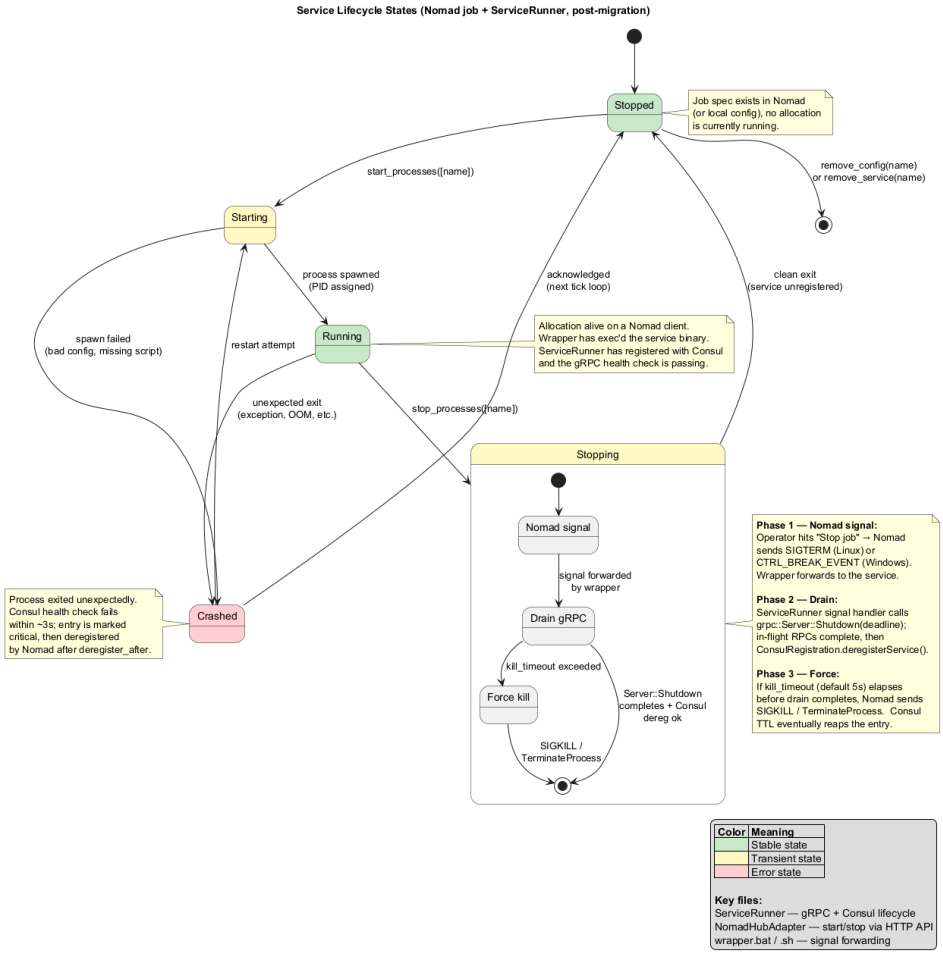


Figure 2.6: Service Process Lifecycle States

Chapter 3

serve_wasm.py

Serve a WASM build directory with COOP/COEP headers.

Multi-threaded Qt WASM uses Web Workers + SharedArrayBuffer. Browsers gate SharedArrayBuffer behind cross-origin isolation:

> Cross-Origin-Opener-Policy: same-origin Cross-Origin-Embedder-Policy: require-corp

Without these the page loads but Qt's threading bootstrap aborts (the WASM module hangs at startup with no visible error). Python's stock `http.server` doesn't set them; this thin subclass does.

Run from the qt_client/ directory: python serve_wasm.py # serves ./build-wasm on :8000 python serve_wasm.py build-wasm # explicit dir PORT=9000 python serve_wasm.py # override port

3.1 Function: main

3.2 Class: COOPCOEPHandler

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.TestService
↪  import COOPCOEPHandler
```

3.2.1 Method: end_headers

Chapter 4

grpc_adapter.py

gRPC inbound adapter for `HelloService`.

Thin mapping layer between the generated `hello_pb2_grpc` stub and the pure-Python domain object. All transport concerns live here; all business rules live in `domain/`.

4.1 Class: HelloGrpcAdapter

Imported by:

```
from
 ↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.hello_servic
 ↪  import HelloGrpcAdapter
```

Maps gRPC calls to `HelloService` methods.

Chapter 5

config.py

Settings for the hello service.

Environment variables are read with the `HELLO_` prefix, e.g.:

HELLO_GRPC_PORT=50051 HELLO_CONSUL_ADDR=http://127.0.0.1:8500 HELLO_GREETING=Hola

`grpc_port` defaults to `0` (OS-assigned), which matches the Nomad + Consul dynamic port allocation pattern.

5.1 Class: Settings

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.hello_servic
↪  import Settings
```

Chapter 6

context.py

Dependency injection for the hello service.

`create_context()` wires configuration, domain logic and adapters together. Keep this file small — it's the only place where concrete classes are chosen, so swapping adapters (e.g. for tests) is a one-line change.

6.1 Function: create_context

6.2 Class: Context

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.hello_servic
↪  import Context
```

Chapter 7

hello_service.py

Pure business logic for the hello service.

No gRPC, no Consul, no I/O — just async Python. This is what unit tests exercise directly, and what the gRPC adapter delegates to.

7.1 Class: HelloService

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.hello_servic
↪  import HelloService
```

Stateless greeting service.

The only configuration is the greeting prefix (e.g. `"Hello"` or `"Bonjour"`), which comes from the service settings.

Chapter 8

main.py

Entry point for the hello service.

Run with:

> cd examples/hello_service python scripts/generate_protos.py # once, to create proto/hello_pb2*.py python main.py

Or via uv:

> uv run python main.py

The service registers itself with Consul on startup and deregisters on a Ctrl+C / SIGTERM / SIGBREAK. With Consul running in dev mode (`consul agent -dev`) you can verify registration with:

> curl http://localhost:8500/v1/catalog/services grpcurl -plaintext -d '{"name": "World"}' <host>:<port> hello.v1.HelloService/Greet

Chapter 9

hello_pb2.py

Generated protocol buffer code.

Chapter 10

hello_pb2_grpc.py

Client and server classes corresponding to protobuf-defined services. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-hello-service-proto-hello-pb2-grpc-add-helloserviceservicer-to-server ==========

10.1 Class: HelloServiceStub

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.hello_servic
↪  import HelloServiceStub
```

HelloService — minimal demo service for the new MicroserviceBase runtime.

Every RPC uses typed request/response messages. The old svc_api_* naming convention is gone; method discovery is handled via gRPC reflection. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-hello-service-proto-hello-pb2-grpc-helloserviceservicer ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.hello_servic
↪  import HelloServiceServicer
```

HelloService — minimal demo service for the new MicroserviceBase runtime.

Every RPC uses typed request/response messages. The old svc_api_* naming convention is gone; method discovery is handled via gRPC reflection. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-hello-service-proto-hello-pb2-grpc-helloserviceservicer-greet ----------

Return a friendly greeting.

10.1.1 Method: Echo

Echo the input back unchanged. Useful for latency measurement.

10.1.2 Method: Tick

Stream N ticks, one per second. Demonstrates server streaming and is how real-time updates (e.g. Cleware switch state changes) will be delivered now that the RabbitMQ fanout exchange is gone. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-hello-service-proto-hello-pb2-grpc-helloservice ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.examples.hello_servic
↪  import HelloService
```

HelloService — minimal demo service for the new MicroserviceBase runtime.

Every RPC uses typed request/response messages. The old svc_api_* naming convention is gone; method discovery is handled via gRPC reflection. microservicebase-microservicemanagergui-dist-win-unpacked-resources-examples-hello-service-proto-hello-pb2-grpc-helloservice-greet ----------

10.1.3 Method: Echo

10.1.4 Method: Tick

Chapter 11

generate_protos.py

Generate Python gRPC stubs from the hello service proto file.

Writes `hello_pb2.py` and `hello_pb2_grpc.py` into the `proto/` folder next to the source `.proto`. Run once after checkout and again whenever you edit `proto/hello.proto`.

11.1 Function: main

Chapter 12

launcher.py

FastAPI Bridge launcher.

Starts the FastAPI Bridge (REST/WS gateway) that connects the MicroserviceManagerGUI to microservices.

Post-migration to gRPC + Consul + Nomad, the bridge no longer requires a RabbitMQ broker to start. If the broker is reachable, the legacy `/api/request` endpoint and the WebSocket service-update stream are enabled; if not, the bridge starts in gRPC/Consul/Nomad-only mode and those legacy paths return a clear "no broker configured" response.

12.1 Function: parse_args

12.2 Function: main

Chapter 13

ClewareAccessHelper.py

13.1 Class: ClewareAccessHelper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import ClewareAccessHelper
```

ClewareAccessHelper class acts as a proxy class in Proxy pattern and forward the request to the real helper depend on platform. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-import-modules-from-paths ----------

Import all modules from given paths.

Arguments:

- `paths`
  / *Condition*: required / *Type*: list /
  List of paths to import modules from.

13.1.1 Method: load_config

Load the user config port name for USB access. Returns: None microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-get-config ----------

Get Cleware ports' config. Returns: Dictionary of ports' setting. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-set-config ----------

Save Cleware ports' config to file. Args: config_json: data to be saved.

Returns: res: saved result.

13.1.2 Method: init_cleware

13.1.3 Method: open_cleware

13.1.4 Method: close_cleware

13.1.5 Method: get_handle

13.1.6 Method: set_value

13.1.7 Method: get_value

13.1.8 Method: set_switch

13.1.9 Method: set_switch_by_port_name

13.1.10 Method: get_switch

13.1.11 Method: get_version

13.1.12 Method: get_usb_type

13.1.13 Method: get_serial_number

13.1.14 Method: valid_ser_number

13.1.15 Method: get_hw_version

13.1.16 Method: is_ampel

13.1.17 Method: iox

13.1.18 Method: get_all_devices_state

Chapter 14

ClewareAccessHelperAbs.py

14.1 Class: ClewareAccessHelperAbs

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import ClewareAccessHelperAbs
```

ClewareAccessHelperAbs acts as a abstract class for the real Helper in Proxy Pattern. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-clewareaccesshelperabs-clewareaccesshelper init-cleware ----------

14.1.1 Method: open_cleware

14.1.2 Method: close_cleware

14.1.3 Method: set_value

14.1.4 Method: get_value

14.1.5 Method: set_switch

14.1.6 Method: get_switch

14.1.7 Method: get_handle

14.1.8 Method: get_version

14.1.9 Method: get_usb_type

14.1.10 Method: get_usb_type

14.1.11 Method: get_serial_number

14.1.12 Method: valid_ser_number

14.1.13 Method: get_hw_version

14.1.14 Method: is_ampel

14.1.15 Method: iox

Chapter 15

ClewareAccessHelperLinux.py

15.1 Class: ClewareAccessHelperLinux

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import ClewareAccessHelperLinux
```

ClewareAccessHelperLinux acts as the real object on Linux platform in Proxy Pattern

- This is the wrapper class for the functions provided by C/C++ source for Linux.
- The C/C++ source for Linux is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN
- The C/C++ source for Linux is modified by Cuong Nguyen (RBVH/ENG22) for support python to load dynamic library on Linux.

15.1.1 Method: init_cleware

15.1.2 Method: open_cleware

15.1.3 Method: close_cleware

15.1.4 Method: get_handle

15.1.5 Method: set_value

15.1.6 Method: get_value

15.1.7 Method: set_switch

15.1.8 Method: get_switch

15.1.9 Method: get_version

15.1.10 Method: get_usb_type

15.1.11 Method: get_serial_number

15.1.12 Method: valid_ser_number

15.1.13 Method: get_hw_version

15.1.14 Method: is_ampel

15.1.15 Method: iox

15.1.16 Method: get_all_sw_state

Chapter 16

ClewareAccessHelperWindows.py

16.1 Class: ClewareAccessHelperWindows

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import ClewareAccessHelperWindows
```

ClewareAccessHelperWindows acts as the real object on Windows platform in Proxy Pattern

- This is the wrapper class for the functions provided by USBaccess.dll.
- The USBaccess.dll is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN

16.1.1 Method: init_cleware

16.1.2 Method: open_cleware

16.1.3 Method: close_cleware

16.1.4 Method: get_handle

16.1.5 Method: set_value

16.1.6 Method: get_value

16.1.7 Method: set_switch

16.1.8 Method: get_switch

16.1.9 Method: get_version

16.1.10 Method: get_usb_type

16.1.11 Method: get_serial_number

16.1.12 Method: valid_ser_number

16.1.13 Method: get_hw_version

16.1.14 Method: is_ampel

16.1.15 Method: iox

Chapter 17

ServiceCleware.py

17.1 Class: ServiceCleware

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import ServiceCleware
```

Service for controlling Cleware devices.

This class extends ServiceBase to provide functionalities specific to managing and controlling Cleware devices.

17.1.1 Method: svc_api_get_all_devices_state

Retrieve the state of all Cleware devices.

Returns:

/ *Type*: dict /
A dictionary containing the states of all Cleware devices.

17.1.2 Method: svc_api_set_switch

Set state for a Cleware device's switch.

Arguments:

- `device_no`
  / *Condition*: required / *Type*: str /
  Cleware device's number.
- `switch_id`
  / *Condition*: required / *Type*: str /
  Switch number to turn on or off.
- `state`
  / *Condition*: required / *Type*: str /
  State of swith to set (on/off).

Returns:

/ *Type*: int /
Return ret code, 1 for succeed, 0 for failure.

17.1.3 Method: notify_updates

Notify updates to the realtime update channel for Cleware devices.

Returns:

(*no returns*)

Chapter 18

ServiceLogger.py

18.1 Class: ServiceLogger

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import ServiceLogger
```

Logger class. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-servicelogger-servicelogger-set-logger ----------

18.1.1 Method: get_config_file

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

18.1.2 Method: log

18.1.3 Method: log_info

18.1.4 Method: log_debug

18.1.5 Method: log_warning

18.1.6 Method: log_error

18.1.7 Method: log_critical

Chapter 19

Utils.py

19.1 Class: Singleton

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import Singleton
```

Class to implement Singleton Design Pattern. This class is used to derive the TTFisClientReal as only a single instance of this class is allowed.

Disabled pyLint Messages: R0903: Too few public methods (%s/%s) Used when class has too few public methods, so be sure it's really worth it.

> This base class implements the Singleton Design Pattern required for the TTFisClientReal. Adding further methods does not make sense.

19.2 Class: Utils

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import Utils
```

Class to implement utilities for supporting development. microservicebase-microservicemanagergui-dist-win-unpacked-resources-python-services-clewareservice-utils-utils-get-all-descendant-classes ----------

Get all descendant classes of a class Args: cls: Input class for finding descendants.

Returns: Array of descendant classes.

19.2.1 Method: get_all_sub_classes

Get all children classes of a class Args: cls: Input class for finding children.

Returns: Array of children classes.

19.2.2 Method: caller_name

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

19.2.3 Method: load_library

Load native library depend on the calling convention. Args: path: library path. is_stdcall: determine if the library's calling convention is stdcall or cdecl.

Returns: Loaded library object.

19.2.4 Method: is_ascii_or_unicode

Check if the string is ascii or unicode Args: str_check: string for checking codecs: encoding type list

Returns: True : if checked string is ascii or unicode False : if checked string is not ascii or unicode

19.2.5 Method: get_registry_parameters

Get service's parameters which stored in registry. Args: service_name: Name of the service. key_name: Name of register key.

Returns: Value of service's parameters

19.2.6 Method: create_registry_parameters

Create service's parameters and store it to registry. Args: service_name: Name of the service. key_name: Name of register key. parameters: Parameters to be stored.

Returns: None

19.3 Class: Job

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.dist.win-unpacked.resources.python.services.Clewa
↪  import Job
```

19.3.1 Method: stop

19.3.2 Method: run

Chapter 20

__main__.py

20.1 Function: main

20.2 Function: signal_handler

Chapter 21

main.py

21.1 Function: main

21.2 Function: signal_handler

Chapter 22

start_bridge.py

Standalone FastAPI UI Bridge launcher.

Starts the FastAPI bridge that exposes microservices via REST API and WebSocket. Can be spawned by the Electron GUI or run directly from the command line.

22.1 Function: parse_args

22.2 Function: main

Chapter 23

start_registry.py

Standalone Service Registry launcher.

Starts the Service Registry that tracks all services, handles registration events, and broadcasts updates to UI clients via a fanout exchange.

23.1 Function: parse_args

23.2 Function: main

Chapter 24

__init__.py

biplist -- a library for reading and writing binary property list files.

Binary Property List (plist) files provide a faster and smaller serialization format for property lists on OS X. This is a library for generating binary plists which can be read by OS X, iOS, or other clients.

The API models the plistlib API, and will call through to plistlib when XML serialization or deserialization is required.

To generate plists with UID values, wrap the values with the Uid object. The value must be an int.

To generate plists with NSData/CFData values, wrap the values with the Data object. The value must be a string.

Date values can only be datetime.datetime objects.

The exceptions InvalidPlistException and NotBinaryPlistException may be thrown to indicate that the data cannot be serialized or deserialized as a binary plist.

Plist generation example:

> from biplist import * from datetime import datetime plist = {'aKey':'aValue', '0':1.322, 'now':datetime.now(), 'list':[1,2,3], 'tuple':('a','b','c') } try: writePlist(plist, "example.plist") except (InvalidPlistException, NotBinaryPlistException), e: print "Something bad happened:", e

Plist parsing example:

> from biplist import * try: plist = readPlist("example.plist") print plist except (InvalidPlistException, NotBinaryPlistException), e: print "Not a plist:", e

24.1 Function: readPlist

Raises NotBinaryPlistException, InvalidPlistException microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---wrapdataobject ==========

24.2 Function: writePlist

24.3 Function: readPlistFromString

24.4 Function: writePlistToString

24.5 Function: is_stream_binary_plist

24.6 Class: Uid

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import Uid
```

Wrapper around integers for representing UID values. This is used in keyed archiving. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---data ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import Data
```

Wrapper around bytes to distinguish Data values. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---invalidplistexception ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import InvalidPlistException
```

Raised when the plist is incorrectly formatted. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---notbinaryplistexception ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import NotBinaryPlistException
```

Raised when a binary plist was expected but not encountered. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistreader ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import PlistReader
```

24.6.1 Method: parse

24.6.2 Method: reset

24.6.3 Method: readRoot

24.6.4 Method: setCurrentOffsetToObjectNumber

24.6.5 Method: beginOffsetProtection

24.6.6 Method: endOffsetProtection

24.6.7 Method: readObject

24.6.8 Method: readContents

24.6.9 Method: readInteger

24.6.10 Method: readReal

24.6.11 Method: readRefs

24.6.12 Method: readArray

24.6.13 Method: readDict

24.6.14 Method: readAsciiString

24.6.15 Method: readUnicode

24.6.16 Method: readDate

24.6.17 Method: readData

24.6.18 Method: readUid

24.6.19 Method: getSizedInteger

Numbers of 8 bytes are signed integers when they refer to numbers, but unsigned otherwise. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---hashablewrapper ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import HashableWrapper
```

24.7 Class: BoolWrapper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import BoolWrapper
```

24.8 Class: FloatWrapper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import FloatWrapper
```

24.9 Class: StringWrapper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import StringWrapper
```

24.9.1 Method: encodingMarker

24.10 Class: PlistWriter

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import PlistWriter
```

24.10.1 Method: reset

24.10.2 Method: positionOfObjectReference

If the given object has been written already, return its position in the offset table. Otherwise, return None. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeroot ----------

Strategy is: - write header - wrap root object so everything is hashable - compute size of objects which will be written - need to do this in order to know how large the object refs will be in the list/dict/set reference lists - write objects - keep objects in writtenReferences - keep positions of object references in referencePositions - write object references with the length computed previously - computer object reference length - write object reference positions - write trailer microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-beginrecursionprotection ----------

24.10.3 Method: endRecursionProtection

24.10.4 Method: wrapRoot

24.10.5 Method: incrementByteCount

24.10.6 Method: computeOffsets

24.10.7 Method: writeObjectReference

Tries to write an object reference, adding it to the references table. Does not write the actual object bytes or set the reference position. Returns a tuple of whether the object was a new reference (True if it was, False if it already was in the reference table) and the new output. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeobject ----------

Serializes the given object to the output. Returns output. If setReferencePosition is True, will set the position the object was written. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeoffsettable ----------

Writes all of the object reference offsets. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-binaryreal ----------

24.10.8 Method: binaryInt

24.10.9 Method: intSize

Returns the number of bytes necessary to store the given integer. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-realsize ----------

Chapter 25

badge.py

25.1 Function: badge_disk_icon

Chapter 26

colors.py

26.1 Function: isAColor

26.2 Function: parseColor

26.3 Class: Color

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import Color
```

26.3.1 Method: to_rgb

26.4 Class: RGB

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import RGB
```

26.4.1 Method: to_rgb

26.5 Class: HSL

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import HSL
```

26.5.1 Method: to_rgb

26.6 Class: HWB

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import HWB
```

26.6.1 Method: to_rgb

26.7 Class: CMYK

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import CMYK
```

26.7.1 Method: to_rgb

26.8 Class: Gray

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import Gray
```

26.8.1 Method: to_rgb

26.9 Class: ColorParser

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import ColorParser
```

26.9.1 Method: skipws

26.9.2 Method: expect

26.9.3 Method: expectEnd

26.9.4 Method: getToken

26.9.5 Method: parseNumber

26.9.6 Method: parseColor

26.9.7 Method: parseRGB

26.9.8 Method: parseHSL

26.9.9 Method: parseHWB

26.9.10 Method: parseCMYK

26.9.11 Method: parseGray

26.9.12 Method: parseValue

26.9.13 Method: parseAngle

Chapter 27

core.py

27.1 Function: build_dmg

27.2 Class: DMGError

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.core
↪  import DMGError
```

Chapter 28

buddy.py

28.1 Class: BuddyError

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy
↪  import BuddyError
```

28.2 Class: Block

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy
↪  import Block
```

28.2.1 Method: close

28.2.2 Method: flush

28.2.3 Method: invalidate

28.2.4 Method: zero_fill

28.2.5 Method: tell

28.2.6 Method: seek

28.2.7 Method: read

28.2.8 Method: write

28.2.9 Method: insert

28.2.10 Method: delete

28.3 Class: Allocator

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy
↪  import Allocator
```

28.3.1 Method: open

28.3.2 Method: close

28.3.3 Method: flush

28.3.4 Method: read

Read data at ‘offset', or raise an exception. ‘size_or_format' may either be a byte count, in which case we return raw data, or a format string for ‘struct.unpack', in which case we work out the size and unpack the data before returning it. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-write ----------

Write data at ‘offset', or raise an exception. ‘data_or_format' may either be the data to write, or a format string for ‘struct.pack', in which case we pack the additional arguments and write the resulting data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-get-block ----------

28.3.5 Method: allocate

Allocate or reallocate a block such that it has space for at least ‘bytes' bytes. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-release ----------

28.3.6 Method: iterkeys

28.3.7 Method: keys

Chapter 29

store.py

29.1 Class: ILocCodec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import ILocCodec
```

29.1.1 Method: encode

29.1.2 Method: decode

29.2 Class: PlistCodec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import PlistCodec
```

29.2.1 Method: encode

29.2.2 Method: decode

29.3 Class: BookmarkCodec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import BookmarkCodec
```

29.3.1 Method: encode

29.3.2 Method: decode

29.4 Class: DSStoreEntry

Imported by:

```
from
 ↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
 ↪  import DSStoreEntry
```

Holds the data from an entry in a `.DS_Store` file. Note that this is not meant to represent the entry itself---i.e. if you change the type or value, your changes will *not* be reflected in the underlying file.

If you want to make a change, you should either use the `DSStore` object's `DSStore.insert` method (which will replace a key if it already exists), or the mapping access mode for `DSStore` (often simpler anyway). microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-read ----------

Read a `.DS_Store` entry from the containing Block microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-byte-length ----------

Compute the length of this entry, in bytes microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-write ----------

Write this entry to the specified Block microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore ==========

Imported by:

```
from
 ↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
 ↪  import DSStore
```

Python interface to a `.DS_Store` file. Works by manipulating the file on the disk---so this code will work with `.DS_Store` files for *very* large directories.

A `DSStore` object can be used as if it was a mapping, e.g.:

```
d['foobar.dat']['Iloc']
```

will fetch the "Iloc" record for "foobar.dat", or raise `KeyError` if there is no such record. If used in this manner, the `DSStore` object will return (type, value) tuples, unless the type is "blob" and the module knows how to decode it.

Currently, we know how to decode "Iloc", "bwsp", "lsvp", "lsvP" and "icvp" blobs. "Iloc" decodes to an (x, y) tuple, while the others are all decoded using `biplist`.

Assignment also works, e.g.:

```
d['foobar.dat']['note'] = ('ustr', u'Hello World!')
```

as does deletion with `del`:

```
del d['foobar.dat']['note']
```

This is usually going to be the most convenient interface, though occasionally (for instance when creating a new `.DS_Store` file) you may wish to drop down to using `DSStoreEntry` objects directly. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-open ----------

Open a `.DS_Store` file; pass either a Python file object, or a filename in the `file_or_name` argument and a file access mode in the `mode` argument. If you are creating a new file using the "w" or "w+" modes, you may also specify a list of entries with which to initialise the file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-flush ----------

Flush any dirty data back to the file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-close ----------

Flush dirty data and close the underlying file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-insert ----------

Insert `entry` (which should be a `DSStoreEntry`) into the B-Tree. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-delete ----------

Delete an item, identified by `filename` and `code` from the B-Tree. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-find ----------

Returns a generator that will iterate over matching entries in the B-Tree.

Chapter 30

alias.py

30.1 Function: encode_utf8

30.2 Function: decode_utf8

30.3 Class: AppleShareInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import AppleShareInfo
```

30.4 Class: VolumeInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import VolumeInfo
```

30.4.1 Method: filesystem_type

30.5 Class: TargetInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import TargetInfo
```

30.6 Class: Alias

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import Alias
```

30.6.1 Method: from_bytes

Construct an `Alias` object given binary Alias data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-alias-alias-for-file ----------

Create an `Alias` that points at the specified file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-alias-alias-to-bytes ----------

Returns the binary representation for this `Alias`.

Chapter 31

bookmark.py

31.1 Function: iteritems

31.2 Class: Data

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↪  import Data
```

31.3 Class: URL

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↪  import URL
```

31.3.1 Method: absolute

Return an absolute URL. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↪  import Bookmark
```

31.3.2 Method: from_bytes

Create a `Bookmark` given byte data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-get ----------

Lookup the value for a given key, returning a default if not present. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-to-bytes ----------

Convert this `Bookmark` to a byte representation. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-for-file ----------

Construct a `Bookmark` for a given file.

Chapter 32

osx.py

32.1 Function: getattrlist

32.2 Function: fgetattrlist

32.3 Function: statfs

32.4 Function: fstatfs

32.5 Class: attrlist

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import attrlist
```

32.6 Class: attribute_set_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import attribute_set_t
```

32.7 Class: fsobj_id_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import fsobj_id_t
```

32.8 Class: timespec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import timespec
```

32.9 Class: attrreference_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import attrreference_t
```

32.10 Class: fsid_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import fsid_t
```

32.11 Class: guid_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import guid_t
```

32.12 Class: kauth_ace

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import kauth_ace
```

32.13 Class: kauth_acl

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import kauth_acl
```

32.14 Class: kauth_filesec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import kauth_filesec
```

32.15 Class: diskextent

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import diskextent
```

32.16 Class: Point

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import Point
```

32.17 Class: Rect

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import Rect
```

32.18 Class: FileInfo

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import FileInfo
```

32.19 Class: FolderInfo

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import FolderInfo
```

32.20 Class: FinderInfo

Imported by:

```
from
↪ MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪ import FinderInfo
```

32.21 Class: vol_capabilities_attr_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import vol_capabilities_attr_t
```

32.22 Class: vol_attributes_attr_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import vol_attributes_attr_t
```

32.23 Class: struct_statfs

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import struct_statfs
```

Chapter 33

utils.py

33.1 Class: UTC

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.utils
↪  import UTC
```

33.1.1 Method: utcoffset

33.1.2 Method: dst

33.1.3 Method: tzname

Chapter 34

launcher.py

FastAPI Bridge launcher.

Starts the FastAPI Bridge (REST/WS gateway) that connects the MicroserviceManagerGUI to microservices.

Post-migration to gRPC + Consul + Nomad, the bridge no longer requires a RabbitMQ broker to start. If the broker is reachable, the legacy `/api/request` endpoint and the WebSocket service-update stream are enabled; if not, the bridge starts in gRPC/Consul/Nomad-only mode and those legacy paths return a clear "no broker configured" response.

34.1 Function: parse_args

34.2 Function: main

Chapter 35

ClewareAccessHelper.py

35.1 Class: ClewareAccessHelper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelpe
↪  import ClewareAccessHelper
```

ClewareAccessHelper class acts as a proxy class in Proxy pattern and forward the request to the real helper depend on platform. microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-import-modules-from-paths ----------

Import all modules from given paths.

Arguments:

- `paths`
  / *Condition*: required / *Type*: list /
  List of paths to import modules from.

35.1.1 Method: load_config

Load the user config port name for USB access. Returns: None microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-get-config ----------

Get Cleware ports' config. Returns: Dictionary of ports' setting. microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelper-clewareaccesshelper-set-config ----------

Save Cleware ports' config to file. Args: config_json: data to be saved.

Returns: res: saved result.

35.1.2 Method: init_cleware

35.1.3 Method: open_cleware

35.1.4 Method: close_cleware

35.1.5 Method: get_handle

35.1.6 Method: set_value

35.1.7 Method: get_value

35.1.8 Method: set_switch

35.1.9 Method: set_switch_by_port_name

35.1.10 Method: get_switch

35.1.11 Method: get_version

35.1.12 Method: get_usb_type

35.1.13 Method: get_serial_number

35.1.14 Method: valid_ser_number

35.1.15 Method: get_hw_version

35.1.16 Method: is_ampel

35.1.17 Method: iox

35.1.18 Method: get_all_devices_state

Chapter 36

ClewareAccessHelperAbs.py

36.1 Class: ClewareAccessHelperAbs

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelpe
↪  import ClewareAccessHelperAbs
```

ClewareAccessHelperAbs acts as a abstract class for the real Helper in Proxy Pattern. microservicebase-microservicemanagergui-python-services-clewareservice-clewareaccesshelperabs-clewareaccesshelperabs-init-cleware
---

36.1.1 Method: open_cleware

36.1.2 Method: close_cleware

36.1.3 Method: set_value

36.1.4 Method: get_value

36.1.5 Method: set_switch

36.1.6 Method: get_switch

36.1.7 Method: get_handle

36.1.8 Method: get_version

36.1.9 Method: get_usb_type

36.1.10 Method: get_usb_type

36.1.11 Method: get_serial_number

36.1.12 Method: valid_ser_number

36.1.13 Method: get_hw_version

36.1.14 Method: is_ampel

36.1.15 Method: iox

Chapter 37

ClewareAccessHelperLinux.py

37.1 Class: ClewareAccessHelperLinux

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelpe
↪  import ClewareAccessHelperLinux
```

ClewareAccessHelperLinux acts as the real object on Linux platform in Proxy Pattern

- This is the wrapper class for the functions provided by C/C++ source for Linux.
- The C/C++ source for Linux is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN
- The C/C++ source for Linux is modified by Cuong Nguyen (RBVH/ENG22) for support python to load dynamic library on Linux.

37.1.1 Method: init_cleware

37.1.2 Method: open_cleware

37.1.3 Method: close_cleware

37.1.4 Method: get_handle

37.1.5 Method: set_value

37.1.6 Method: get_value

37.1.7 Method: set_switch

37.1.8 Method: get_switch

37.1.9 Method: get_version

37.1.10 Method: get_usb_type

37.1.11 Method: get_serial_number

37.1.12 Method: valid_ser_number

37.1.13 Method: get_hw_version

37.1.14 Method: is_ampel

37.1.15 Method: iox

37.1.16 Method: get_all_sw_state

Chapter 38

ClewareAccessHelperWindows.py

38.1 Class: ClewareAccessHelperWindows

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ClewareAccessHelpe
↪  import ClewareAccessHelperWindows
```

ClewareAccessHelperWindows acts as the real object on Windows platform in Proxy Pattern

- This is the wrapper class for the functions provided by USBaccess.dll.
- The USBaccess.dll is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN

38.1.1 Method: init_cleware

38.1.2 Method: open_cleware

38.1.3 Method: close_cleware

38.1.4 Method: get_handle

38.1.5 Method: set_value

38.1.6 Method: get_value

38.1.7 Method: set_switch

38.1.8 Method: get_switch

38.1.9 Method: get_version

38.1.10 Method: get_usb_type

38.1.11 Method: get_serial_number

38.1.12 Method: valid_ser_number

38.1.13 Method: get_hw_version

38.1.14 Method: is_ampel

38.1.15 Method: iox

Chapter 39

ServiceCleware.py

39.1 Class: ServiceCleware

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ServiceCleware
↪  import ServiceCleware
```

Service for controlling Cleware devices.

This class extends ServiceBase to provide functionalities specific to managing and controlling Cleware devices.

39.1.1 Method: svc_api_get_all_devices_state

Retrieve the state of all Cleware devices.

Returns:

/ *Type*: dict /
A dictionary containing the states of all Cleware devices.

39.1.2 Method: svc_api_set_switch

Set state for a Cleware device's switch.

Arguments:

- `device_no`
  / *Condition*: required / *Type*: str /
  Cleware device's number.
- `switch_id`
  / *Condition*: required / *Type*: str /
  Switch number to turn on or off.
- `state`
  / *Condition*: required / *Type*: str /
  State of swith to set (on/off).

Returns:

/ *Type*: int /
Return ret code, 1 for succeed, 0 for failure.

39.1.3 Method: notify_updates

Notify updates to the realtime update channel for Cleware devices.

Returns:

(*no returns*)

Chapter 40

ServiceLogger.py

40.1 Class: ServiceLogger

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.ServiceLogger
↪  import ServiceLogger
```

Logger class. microservicebase-microservicemanagergui-python-services-clewareservice-servicelogger-servicelogger-set-logger ----------

40.1.1 Method: get_config_file

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

40.1.2 Method: log

40.1.3 Method: log_info

40.1.4 Method: log_debug

40.1.5 Method: log_warning

40.1.6 Method: log_error

40.1.7 Method: log_critical

Chapter 41

Utils.py

41.1 Class: Singleton

Imported by:

```
from MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.Utils
↪  import Singleton
```

Class to implement Singleton Design Pattern. This class is used to derive the TTFisClientReal as only a single instance of this class is allowed.

Disabled pyLint Messages: R0903: Too few public methods (%s/%s) Used when class has too few public methods, so be sure it's really worth it.

> This base class implements the Singleton Design Pattern required for the TTFisClientReal. Adding further methods does not make sense.

41.2 Class: Utils

Imported by:

```
from MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.Utils
↪  import Utils
```

Class to implement utilities for supporting development. microservicebase-microservicemanagergui-python-services-clewareservice-utils-utils-get-all-descendant-classes ----------

Get all descendant classes of a class Args: cls: Input class for finding descendants.

Returns: Array of descendant classes.

41.2.1 Method: get_all_sub_classes

Get all children classes of a class Args: cls: Input class for finding children.

Returns: Array of children classes.

41.2.2 Method: caller_name

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

41.2.3 Method: load_library

Load native library depend on the calling convention. Args: path: library path. is_stdcall: determine if the library's calling convention is stdcall or cdecl.

Returns: Loaded library object.

41.2.4 Method: is_ascii_or_unicode

Check if the string is ascii or unicode Args: str_check: string for checking codecs: encoding type list

Returns: True : if checked string is ascii or unicode False : if checked string is not ascii or unicode

41.2.5 Method: get_registry_parameters

Get service's parameters which stored in registry. Args: service_name: Name of the service. key_name: Name of register key.

Returns: Value of service's parameters

41.2.6 Method: create_registry_parameters

Create service's parameters and store it to registry. Args: service_name: Name of the service. key_name: Name of register key. parameters: Parameters to be stored.

Returns: None

41.3 Class: Job

Imported by:

```
from MicroserviceBase.MicroserviceManagerGUI.python.services.ClewareService.Utils
↪  import Job
```

41.3.1 Method: stop

41.3.2 Method: run

Chapter 42

__main__.py

42.1 Function: main

42.2 Function: signal_handler

Chapter 43

main.py

43.1 Function: main

43.2 Function: signal_handler

Chapter 44

start_bridge.py

Standalone FastAPI UI Bridge launcher.

Starts the FastAPI bridge that exposes microservices via REST API and WebSocket. Can be spawned by the Electron GUI or run directly from the command line.

44.1 Function: parse_args

44.2 Function: main

Chapter 45

start_registry.py

Standalone Service Registry launcher.

Starts the Service Registry that tracks all services, handles registration events, and broadcasts updates to UI clients via a fanout exchange.

45.1 Function: parse_args

45.2 Function: main

Chapter 46

eventbus_config.py

46.1 Class: EventBusConfig

Imported by:

```
from MicroserviceBase.adapters.config.eventbus_config import EventBusConfig
```

Configuration for EventBusClient-based adapters.

Wraps a JSONP config file path used by `EventBusClient.from_config_sync()`.

Attributes:

- `config_path`
  / *Type*: str / *Default*: None /
  Path to the JSONP file consumed by `EventBusClient.PluginLoader`.

46.1.1 Method: load_config

Load and return the parsed config as a dict.

Requires EventBusClient to be installed.

Returns:

/ *Type*: dict /
Parsed configuration dictionary.

46.1.2 Method: to_config_source

Load config and return as a dict with optional field overrides.

Useful for creating derivative clients (e.g., registry or fanout) that share connection settings but differ in exchange configuration.

Arguments:

- `overrides`
  / *Condition*: optional / *Type*: keyword arguments /
  Fields to override in the loaded config (e.g., exchange_name, exchange_handler).

Returns:

/ *Type*: dict /
Config dictionary with overrides applied.

Chapter 47

rabbitmq_config.py

47.1 Class: RabbitMQConfig

Imported by:

```
from MicroserviceBase.adapters.config.rabbitmq_config import RabbitMQConfig
```

Configuration for connecting to RabbitMQ.

47.1.1 Method: to_connection_params

Convert to pika ConnectionParameters kwargs.

Returns:

dict suitable for pika.ConnectionParameters(**kwargs).

47.1.2 Method: from_cmd_args

Parse RabbitMQ config from command-line arguments and environment variables.

Arguments:

- `cmd_args`
  / *Condition*: optional / *Type*: list /
  List of CLI arguments, or None for sys.argv.
- `service_name`
  / *Condition*: optional / *Type*: str / *Default*: 'Service' /
  Name of the service (used in help text).

Returns:

RabbitMQConfig instance.

Chapter 48

__init__.py

gRPC bridge adapter — dynamic method discovery and invocation.

Exposes `GrpcReflectClient` used by the FastAPI bridge to let the GUI enumerate and call methods on any gRPC service registered in Consul, without requiring the bridge to know the service's `.proto` ahead of time. All type information is obtained at runtime via the standard gRPC Server Reflection Protocol.

Chapter 49

local_proto_client.py

Compatibility re-export for LocalProtoClient.

QConnectBase (QConnectBase.grpc.grpc_client) imports the fallback client as:

```
from MicroserviceBase.adapters.grpc_bridge.local_proto_client import (
    LocalProtoClient,
)
```

The class itself lives in `reflect_client` alongside `GrpcReflectClient` — they share a lot of internal helpers and keeping them in one file keeps imports cheap. This thin module just re-exports the class under the legacy path so QConnectBase's `try: import` succeeds and `_HAS_LOCAL_PROTO` flips to True.

If you start a new caller, prefer the canonical import:

from MicroserviceBase.adapters.grpc_bridge.reflect_client import (LocalProtoClient,
)

Chapter 50

reflect_client.py

Dynamic gRPC client using the server reflection protocol.

Services that enable the standard reflection service (which `MicroserviceBase.runtime.ServiceRunner` does automatically) can be called without their proto files being available locally — the reflection service returns their `FileDescriptorProto` bytes and we feed them into a `google.protobuf.descriptor_pool.DescriptorPool` to build dynamic message classes.

This keeps the GUI bridge completely decoupled from individual services: add a new service, and it just appears in the GUI with its methods browsable.

When the server **doesn't** ship reflection (e.g. built against vcpkg's grpc port which doesn't include `grpc++_reflection`), the bridge can fall back to `LocalProtoClient` which compiles `.proto` files from disk via `grpc_tools.protoc`. Same public API, no server help required.

Only **unary-unary** RPCs are supported at this stage. Streaming methods are detected and reported but cannot yet be invoked.

50.1 Class: GrpcReflectError

Imported by:

```
from MicroserviceBase.adapters.grpc_bridge.reflect_client import GrpcReflectError
```

Any failure from the reflection client — network, parse, or invoke.

Attributes:

- `is_unimplemented`
  / *Type*: bool /
  `True` when the failure came from a reflection RPC that the server returned `UNIMPLEMENTED` for — the canonical signal that the server wasn't built with reflection support. Bridge code uses this to decide whether to fall back to `LocalProtoClient`.

50.2 Class: GrpcReflectClient

Imported by:

```
from MicroserviceBase.adapters.grpc_bridge.reflect_client import GrpcReflectClient
```

Talk to a gRPC server using only its address + the reflection API.

Usage:

```
client = GrpcReflectClient("127.0.0.1:50051")
services = client.list_services()
methods = client.list_methods("hello.v1.HelloService")
```

```
result = client.call_unary(
    "hello.v1.HelloService", "Greet", '{"name": "World"}'
)
```

The instance is **not** thread-safe. Create one per request on the bridge side.

50.2.1 Method: close

Close the underlying gRPC channel. Safe to call multiple times.

Returns:

(*no returns*)

50.2.2 Method: list_services

Return all fully-qualified service names exposed by the target, excluding the built-in reflection / health services.

Returns:

- `services`
  / *Type*: List[str] /
  Sorted-by-server fully-qualified service names (e.g. `"hello.v1.HelloService"`). Empty list when the server exposes only built-in services.

50.2.3 Method: list_methods

Return method descriptors for `full_service_name`.

Arguments:

- `full_service_name`
  / *Condition*: required / *Type*: str /
  Fully-qualified service name (e.g. `"hello.v1.HelloService"`).

Returns:

- `methods`
  / *Type*: List[Dict[str, Any]] /
  One dict per RPC, with keys `name`, `input_type`, `output_type`, `client_streaming`, `server_streaming`, `input_fields` (shallow field list), and `input_skeleton` (JSON skeleton with default values).

50.2.4 Method: call_method

Invoke a unary-unary or server-streaming method.

- **Unary-unary** → returns {`"streaming": False, "result": <dict>`}.
- **Server-streaming** → reads up to `max_events` events from the stream (or until `max_seconds` elapses), then cancels the RPC and returns {`"streaming": True, "events": [<dict>, ...], "truncated": bool`}.
- **Client- or bidi-streaming** → raises `GrpcReflectError`.

Client-streaming and bidirectional streaming need a different UI pattern (incremental request sends + cancel button) and are intentionally rejected here.

Arguments:

- `full_service_name`
  / *Condition*: required / *Type*: str /
  Fully-qualified service name (e.g. `"hello.v1.HelloService"`).

- `method_name`
  / *Condition*: required / *Type*: str /
  Method name within the service (e.g. `"Greet"`).
- `args_json`
  / *Condition*: required / *Type*: str /
  JSON-encoded request message. Empty string is allowed for methods with no required fields.
- `max_events`
  / *Condition*: optional / *Type*: int / *Default*: 100 /
  Server-streaming only: maximum events to collect before cancelling.
- `max_seconds`
  / *Condition*: optional / *Type*: float / *Default*: 15.0 /
  Server-streaming only: maximum wall-clock seconds before cancelling.

Returns:

- `result`
  / *Type*: Dict[str, Any] /
  See the description above for the per-mode shape.

50.2.5 Method: call_unary

Backwards-compat alias for `call_method` that unwraps unary results to match the older shape.

For unary-unary calls returns the response dict directly. For server-streaming calls returns the same shape as `call_method` (`{"streaming": True, ...}`) so callers can still detect streams.

Arguments:

- `args, kwargs`
  / *Condition*: forwarded / *Type*: Any /
  Forwarded verbatim to `call_method`.

Returns:

- `result`
  / *Type*: Dict[str, Any] /
  Response dict (unary) or full streaming envelope.

50.3 Class: LocalProtoClient

Imported by:

```
from MicroserviceBase.adapters.grpc_bridge.reflect_client import LocalProtoClient
```

Drop-in replacement for `GrpcReflectClient` when the server has no reflection support.

Builds the descriptor pool by compiling local `.proto` files via `grpc_tools.protoc` instead of asking the server. Has the same public methods (`list_services`, `list_methods`, `call_method`, `call_unary`) so callers can swap one for the other.

The client owns a gRPC channel to *target* for the actual invocations — only schema discovery is local.

Usage: client = LocalProtoClient(”127.0.0.1:50051”, proto_files=[”proto/hello.proto”], # explicit
) # or client = LocalProtoClient.from_search_paths(”127.0.0.1:50051”, search_paths=[”./proto”, ”C:/projects”],
)

50.3.1 Method: from_search_paths

Build a client by globbing `**/*.proto` under each search path.

Empty/missing dirs are silently skipped. Each search path is also added as a protoc `-I` include path so cross-file imports resolve.

Sub-directories matching `_EXCLUDE_FRAGMENTS` (`build/`, `vcpkg_installed/`, `node_modules/`, ...) are pruned so the glob doesn't pick up vendor copies of well-known types or duplicated stubs that would break `protoc`. microservicebase-adapters-grpc-bridge-reflect-client-localprotoclient-close ----------

50.3.2 Method: list_services

Return all fully-qualified service names found in the loaded proto files, excluding built-in reflection / health services. microservicebase-adapters-grpc-bridge-reflect-client-localprotoclient-list-methods ----------

Same shape as `GrpcReflectClient.list_methods`. microservicebase-adapters-grpc-bridge-reflect-client-localprotoclient-call-method ----------

Same semantics as `GrpcReflectClient.call_method`. microservicebase-adapters-grpc-bridge-reflect-client-localprotoclient-call-unary ----------

Chapter 51

local_hub_manager.py

Local Hub Manager -- manages a local ProcessHub instance lifecycle.

Wraps ProcessHub APIs (lazy import) so the MicroserviceManager GUI can start/stop a ProcessHub on the local machine, manage processes, and optionally join a fleet as an agent.

51.1 Class: LocalHubManager

Imported by:

```
from MicroserviceBase.adapters.local_hub.local_hub_manager import LocalHubManager
```

Manages a local ProcessHub instance lifecycle.

51.1.1 Method: start_hub

Start a local ProcessHub server.

Arguments:

- `mode`
  / *Condition*: optional / *Type*: str / *Default*: 'standalone' /
  "standalone", "agent" (join fleet), or "orchestrator".
- `xpub_port`
  / *Condition*: optional / *Type*: int / *Default*: 5555 /
  ZMQ XPUB port for the broker.
- `xsub_port`
  / *Condition*: optional / *Type*: int / *Default*: 5556 /
  ZMQ XSUB port for the broker.
- `orchestrator_url`
  / *Condition*: optional / *Type*: str /
  Fleet orchestrator ZMQ address (agent mode).
- `hub_id`
  / *Condition*: optional / *Type*: str /
  Hub identifier (agent/orchestrator mode).
- `hub_name`
  / *Condition*: optional / *Type*: str /
  Human-readable hub name (agent/orchestrator mode).

- `process_config`
  / *Condition*: optional / *Type*: dict /
  Dict of {name: config} for processes.
- `fleet_api_port`
  / *Condition*: optional / *Type*: int / *Default*: 2510 /
  FleetWebAPI port (orchestrator mode).
- `health_timeout`
  / *Condition*: optional / *Type*: float / *Default*: 30.0 /
  Health check timeout in seconds (orchestrator mode).

Returns:

/ *Type*: dict /
Status dict.

51.1.2 Method: stop_hub

Stop the local hub.

51.1.3 Method: get_status

Get current hub status.

51.1.4 Method: start_processes

Start named processes.
Arguments:

- `names`
  / *Condition*: required / *Type*: list /
  List of process names to start.

51.1.5 Method: stop_processes

Stop named processes.
Arguments:

- `names`
  / *Condition*: required / *Type*: list /
  List of process names to stop.
- `force`
  / *Condition*: optional / *Type*: bool / *Default*: False /
  If True, force-kill the processes.

51.1.6 Method: get_config

Get all process configurations.

51.1.7 Method: add_config

Add a new process configuration.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.
- `config`
  / *Condition*: required / *Type*: dict /
  Process configuration dict.

51.1.8 Method: update_config

Update an existing process configuration.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.
- `config`
  / *Condition*: required / *Type*: dict /
  Updated process configuration dict.

51.1.9 Method: remove_config

Remove a process configuration.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.

51.1.10 Method: remove_service

Remove a service -- delete config and managed service folder.

Only deletes the folder if it lives inside the managed `<config_dir>/services/` directory (never touches external paths). Stops the process first if it is still running.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Service name.

51.1.11 Method: get_service_log

Read the last *tail* lines of a process log file.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.

- `tail`
  / *Condition*: optional / *Type*: int / *Default*: 100 /
  Number of lines to read from the tail.

51.1.12 Method: get_services_dir

Return absolute path to `<config_dir>/services/`, creating it if needed.

51.1.13 Method: import_service

Import a microservice from a folder or base64-encoded ZIP.

Validates structure, copies files into `<config_dir>/services/{name}/`, and creates a hub config entry using the `${python}` placeholder.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Service name.
- `source_path`
  / *Condition*: optional / *Type*: str /
  Path to the source directory to import from.
- `zip_data`
  / *Condition*: optional / *Type*: str /
  Base64-encoded ZIP data to import from.
- `wait_time`
  / *Condition*: optional / *Type*: float / *Default*: 1.0 /
  Seconds to wait after starting before checking process health.

Returns:

/ *Type*: dict /
`{"success": bool, "message": str, "warnings": [...]}`.

51.1.14 Method: reset

Reset the hub (stop processes, clear connections).

Chapter 52

service_executor.py

Service-aware process executor.

Wraps a `SimpleExecutor` and attempts graceful RPC shutdown (`svc_api_shutdown`) before falling back to signal-based stop.

52.1 Class: ServiceExecutor

Imported by:

```
from MicroserviceBase.adapters.local_hub.service_executor import ServiceExecutor
```

Process executor that sends an RPC shutdown to MicroserviceBase services.

Delegates all operations to a wrapped `SimpleExecutor`. On `stop()`, it first tries to send `svc_api_shutdown` via RabbitMQ to the service's queue. If the service exits within `shutdown_timeout` seconds the process is considered stopped. Otherwise it falls back to the delegate's signal-based stop (CTRL_BREAK_EVENT on Windows, SIGTERM on Linux).

52.1.1 Method: start

Start a named process using the given config.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.
- `config`
  / *Condition*: required / *Type*: dict /
  Process configuration dict with 'script', 'args', 'cwd', etc.

52.1.2 Method: get_log_path

Public accessor for log file path.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.

52.1.3 Method: read_log

Public method to read process log. Returns the text content.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.
- `tail`
  / *Condition*: optional / *Type*: int / *Default*: 100 /
  Number of lines to read from the tail.

52.1.4 Method: is_running

Check if a named process is running.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.

52.1.5 Method: get_pid

Get the PID of a named process.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.

52.1.6 Method: stop

Stop a named process, attempting RPC shutdown first.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  Process name.
- `force`
  / *Condition*: optional / *Type*: bool / *Default*: False /
  If True, skip graceful shutdown and force-kill.

Chapter 53

nomad_client.py

53.1 Class: NomadAPIError

Imported by:

```
from MicroserviceBase.adapters.nomad_hub.nomad_client import NomadAPIError
```

Raised when the Nomad API returns a non-2xx response.

Attributes:

- `status_code`
  / *Type*: int /
  HTTP status code returned by Nomad (or `0` for transport errors).
- `url`
  / *Type*: str /
  The URL that was being requested when the error occurred.

53.2 Class: NomadClient

Imported by:

```
from MicroserviceBase.adapters.nomad_hub.nomad_client import NomadClient
```

HTTP client for the HashiCorp Nomad v1 API.

Uses only `urllib` from the standard library — no external dependencies. Supports ACL token authentication and blocking queries (long-poll).

53.2.1 Method: agent_self

GET /v1/agent/self — verify agent connectivity. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-list-jobs ----------

GET /v1/jobs — list all jobs (optional prefix filter). microservicebase-adapters-nomad-hub-nomad-client-nomadclient-get-job ----------

GET /v1/job/:id — read full job specification. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-register-job ----------

POST /v1/jobs — register (create or update) a job. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-parse-hcl ----------

POST /v1/jobs/parse — convert an HCL job spec to JSON.

Used by the GUI "Submit Job" dialog so users can paste HCL directly instead of JSON. Nomad's own `nomad job run` does the same under the hood.

Returns the parsed job dict ready to be wrapped in `{"Job": ...}` and POSTed to `/v1/jobs`. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-stop-job ----------

DELETE /v1/job/:id — stop a job. purge=True removes it entirely. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-get-allocations ----------

GET /v1/job/:id/allocations — list allocations for a job. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-get-allocation ----------

GET /v1/allocation/:id — read a single allocation. microservicebase-adapters-nomad-hub-nomad-client-nomadclient-get-logs ----------

GET /v1/client/fs/logs/:alloc_id — read task logs.

Returns plain text when `plain=True`, otherwise JSON frames.

53.2.2 Method: list_jobs_blocking

GET /v1/jobs with blocking query.

Blocks until the job list changes or `wait` expires. Uses the `X-Nomad-Index` from the previous call.

Chapter 54

nomad_hub_adapter.py

54.1 Class: NomadHubAdapter

Imported by:

```
from MicroserviceBase.adapters.nomad_hub.nomad_hub_adapter import NomadHubAdapter
```

`HubManagerPort` implementation that manages services as Nomad jobs.

Each service config is translated into a Nomad job spec using the `raw_exec` driver (Python processes, no Docker required). Method contracts are inherited from `HubManagerPort` — refer to that class's docstrings for the per-method `**Arguments:**` and `**Returns:**` shapes.

Behaviour notes specific to this adapter:

- `start_hub` does *not* start a Nomad agent; it verifies connectivity to an already-running Nomad server and starts the background status-poll thread.
- `stop_hub` stops the poll thread but leaves Nomad jobs running.
- A background poll thread (`_poll_loop`) calls `on_status_change(status)` whenever the status snapshot from `get_status()` changes.

54.1.1 Method: start_hub

54.1.2 Method: stop_hub

54.1.3 Method: get_status

54.1.4 Method: start_processes

54.1.5 Method: stop_processes

54.1.6 Method: get_config

54.1.7 Method: add_config

54.1.8 Method: update_config

54.1.9 Method: remove_config

54.1.10 Method: get_service_log

54.1.11 Method: import_service

54.1.12 Method: remove_service

54.1.13 Method: reset

Chapter 55

amqp_registry_adapter.py

55.1 Class: AMQPRegistryAdapter

Imported by:

```
from MicroserviceBase.adapters.registry.amqp_registry_adapter import
↪  AMQPRegistryAdapter
```

AMQP implementation of the ServiceRegistryPort interface.
Uses RabbitMQ topic exchange for service registration events and fanout exchange for realtime update broadcasts.

55.1.1 Method: register

Register a service by publishing to the service_information exchange.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

55.1.2 Method: unregister

Unregister a service by publishing to the service_information exchange.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

55.1.3 Method: subscribe_to_events

Subscribe to service registration events.
Runs in the calling thread (blocking). Typically called from a daemon thread.
Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, properties, body).

55.1.4 Method: notify_update

Broadcast services update to the realtime update fanout exchange.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  dict of all current service information.

55.1.5 Method: subscribe_to_updates

Subscribe to the realtime update fanout exchange.

Unlike `subscribe_to_events` (which listens for individual on/off events), this subscribes to the full services dict broadcast by the Registry after each change. Uses an exclusive temporary queue to avoid competing consumers.

Runs in the calling thread (blocking). Typically called from a daemon thread.

Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(services_info_dict) called with the full services dict.

55.1.6 Method: get_update_channel_name

Return the realtime update exchange name.

55.1.7 Method: cleanup_update_exchange

Delete the realtime update exchange (for cleanup on shutdown).

55.1.8 Method: cleanup

Close event subscription resources and clean up the update exchange.

Chapter 56

eventbus_registry_adapter.py

56.1 Class: EventBusRegistryAdapter

Imported by:

```
from MicroserviceBase.adapters.registry.eventbus_registry_adapter import
↪  EventBusRegistryAdapter
```

EventBusClient implementation of the ServiceRegistryPort interface.

Uses EventBusClient with TopicExchangeHandler for service registration events and a FanoutExchangeHandler for realtime update broadcasts.

Both clients are created from the same JSONP config with exchange overrides via `EventBusConfig.to_config_source()`.

56.1.1 Method: register

Register a service by publishing to the service_information exchange.

Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

56.1.2 Method: unregister

Unregister a service by publishing to the service_information exchange.

Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  dict containing service metadata.

56.1.3 Method: subscribe_to_events

Subscribe to service registration events.

Blocks the calling thread. Typically called from a daemon thread. The handler receives the pika-style callback signature (ch, method, properties, body) for backward compatibility.

Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, properties, body).

56.1.4 Method: notify_update

Broadcast services update to the realtime update fanout exchange.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  dict of all current service information.

56.1.5 Method: get_update_channel_name

Return the realtime update exchange name.

56.1.6 Method: cleanup

Close all EventBusClient connections.

Chapter 57

__init__.py

Service scaffold generator.

Generates complete project directories for new microservices based on user selections (language, GUI type, infrastructure). Used by the Service Creator wizard in MicroserviceManagerGUI.

Supported combinations:

> Language GUI type Output -------- -------- ------Python none Async gRPC service (domain + adapter + proto) Python html Same + GUIs/service.html + service.js C++ none CMake project (domain + adapter + proto + build scripts) C++ qml Same + QML UI + preview app + stubs C++ wasm Same + Qt Widgets WASM UI + build_wasm script C++ widget Same + Qt Widgets desktop UI

All combinations optionally include:

- Nomad job spec (.nomad.hcl)
- Build/deploy scripts
- README.md

Chapter 58

_template_loader.py

Tiny loader for the templates/ tree shipped alongside this package.

Each cpp_tmpl / python_tmpl helper that used to embed a multi-kilobyte Python f-string now reads its template from a .tmpl file under templates/<lang>/<cluster>/<name>.tmpl.

The template syntax is a custom `string.Template` subclass that uses `@@{var}` instead of `$var`. Why: the default `$` delimiter conflicts with CMake (`${VAR}`) and bash (`$VAR`), and a single `@` conflicts with Doxygen (`@param`, `@return`, `@brief`). Doubling to `@@` dodges every one of these — single `@` and single `$` both stay as literal text.

Placeholder forms:

- `@@{var}` — braced, recommended (always unambiguous)
- `@@var` — unbraced (technically supported, but `@@{var}` is the convention because it can't be confused with following identifier chars in the source)
- `@@@@` — escape for a literal `@@`

Unknown placeholders raise `KeyError` so missing variables fail fast at generation time instead of shipping `@@{placeholder}` text to disk.

Loaded templates are cached in process for the lifetime of the interpreter — they're tiny and they don't change at runtime.

58.1 Function: load_template

Render a template under templates/ with @@{var} substitution.

rel_path is forward-slash relative to templates/ (e.g. `"cpp/server/main.cpp.tmpl"`). Unknown placeholders raise `KeyError` so missing variables fail fast at generation time instead of shipping `@@{placeholder}` text to disk. microservicebase-adapters-scaffold--template-loader-load-raw ==========

Read a template file verbatim, skipping @@{var} substitution.

Use for files that contain literal @@ sequences which would otherwise be mistaken for placeholders — most commonly unified diffs / patches where @@ -L,N +L,N @@ is the hunk-header syntax. microservicebase-adapters-scaffold--template-loader--attemplate ==========

Imported by:

```
from MicroserviceBase.adapters.scaffold._template_loader import _AtTemplate
```

`string.Template` subclass using `@@{var}` instead of `$var`.

Chapter 59

cpp_tmpl.py

C++ service scaffold templates.

Generates CMake-based projects for gRPC microservices using the MicroserviceBase C++ runtime (ServiceRunner + ConsulRegistration). Supports four GUI variants: none, qml, wasm, widget.

59.1 Function: generate

Render every file for a C++ gRPC service scaffold.

Produces a fully buildable project tree: CMakeLists, `main.cpp`, `Settings.h`, domain stubs, gRPC adapter, env-var scripts (`set_env.bat` / `.sh` / MinGW / MSYS2 / MSVC variants), build scripts (`build_deploy*.bat`), proto-stub generator scripts, and optionally Qt UI files (QML / WASM / Widget) per `spec.gui_type`.

Internal `_xxx` helpers in this module each return one file's text verbatim — they are not meant to be called directly; `generate` is the single public entry point.

Arguments:

- `spec`
  / *Condition*: required / *Type*: ScaffoldSpec /
  Scaffold specification (service name, methods, GUI type, client / server gRPC kind, toolchain hints, ...).

Returns:

- `files`
  / *Type*: Dict[str, str] /
  Map of relative-path-string → file-contents-string for every file the scaffold should write. The caller (`adapters/scaffold/generator.py`) writes them to `output_path/service_name/` or zips them for download.

59.2 Function: generate_monorepo

Emit a single-project scaffold with one executable per service.

All services share one proto (written verbatim from spec.proto_content_override or regenerated), one CMakeLists.txt with N add_executable() calls, and a single build_deploy.bat that drops every .exe into dist/.

services is a list of ServiceBlock (name + methods). v1: C++ only, no client subproject, no Qt widget GUI per service. microservicebase-adapters-scaffold-cpp-tmpl-generate-multi-proto ==========

Emit a multi-proto / single-binary scaffold (vehicle-example pattern).

Each service in `services` becomes:

- its own `.proto` file under `proto/` (with its own package),
- its own `domain/` + `adapters/api/` source folder,

and ALL services are registered on one grpc::ServerBuilder by the project's single `src/main.cpp`.

v1: C++ only. Borrows monorepo CMake auto-detection + set_env / build script infrastructure unchanged.

Chapter 60

generator.py

Top-level scaffold generator — dispatches to language-specific modules.

The entry point `generate_scaffold` returns a dict of {`relative_path: content_string`} that the bridge endpoint can write to disk or package into a ZIP.

60.1 Function: generate_scaffold

Return {`path: content`} for every file in the scaffold.

Paths are relative to the project root (e.g. `src/main.cpp`). microservicebase-adapters-scaffold-generator-methodparam ==========

Imported by:

```
from MicroserviceBase.adapters.scaffold.generator import MethodParam
```

60.2 Class: MethodSpec

Imported by:

```
from MicroserviceBase.adapters.scaffold.generator import MethodSpec
```

60.3 Class: ServiceBlock

Imported by:

```
from MicroserviceBase.adapters.scaffold.generator import ServiceBlock
```

One gRPC service entry inside a multi-service scaffold.

Used by two layouts:

- **`monorepo` - N services share ONE .proto, each builds its own** executable. proto_file is ignored; the project's main .proto holds all services.
- **`multi_proto` - N services come from N separate .protos but compile** into ONE executable that registers them all on the same grpc::ServerBuilder. Each service carries its own `proto_file` (defaults to `<snake_name>.proto` when empty).

60.4 Class: ScaffoldSpec

Imported by:

```
from MicroserviceBase.adapters.scaffold.generator import ScaffoldSpec
```

Everything the wizard collects across all steps. microservicebase-adapters-scaffold-generator-scaffoldspec-snake-name ----------

CamelCase → snake_case, keeping runs of capitals together.

`MyService` → `my_service`, `PPSService` → `pps_service`, `XMLParser` → `xml_parser`. microservicebase-adapters-scaffold-generator-scaffoldspec-proto-package ----------

60.4.1 Method: proto_namespace

C++ namespace form of proto_package (dots → ::). microservicebase-adapters-scaffold-generator-scaffoldspec-env-prefix ----------

Chapter 61

python_tmpl.py

Python service scaffold templates.

Generates the full project structure for a Python gRPC microservice using the MicroserviceBase async runtime (ServiceRunner + Consul).

61.1 Function: generate

Render every file for a Python single-service gRPC scaffold.

Produces `main.py`, `config.py` (Pydantic `BaseServiceSettings` subclass), `context.py` (composition root), `pyproject.toml`, domain stubs, gRPC adapter, proto-stub generator script, and optionally HTML/JS GUI files when `spec.gui_type == "html"`.

For multi-service / monorepo layouts use `generate_monorepo`.

Arguments:

- `spec`
  / *Condition*: required / *Type*: ScaffoldSpec /
  Scaffold specification.

Returns:

- `files`
  / *Type*: Dict[str, str] /
  Map of relative-path → file-contents for every file the scaffold should write.

61.2 Function: generate_monorepo

Render a Python monorepo scaffold: one project, N services, N `run_<svc>` console-script entry points.

Used when the user picks the `monorepo` layout in the Service Creator wizard or sets `monorepo: true` in the CLI config. Each service gets its own `main.py`, `config.py`, `context.py`, domain + adapter stubs, and Nomad job spec; `pyproject.toml` lists all entry points so `pip install .` produces N CLI commands.

Arguments:

- `spec`
  / *Condition*: required / *Type*: ScaffoldSpec /
  Outer (project-level) scaffold specification.
- `services`
  / *Condition*: required / *Type*: list /
  List of per-service `ScaffoldSpec`-like objects. Each entry's `service_name`, `methods`, and `proto_package` drive its own generated files.

Returns:

- `files`
  / *Type*: Dict[str, str] /
  Map of relative-path → file-contents for the entire monorepo tree.

Chapter 62

robot_tmpl.py

Generate Robot Framework resource files from .proto files.

For each `service` declared in any `.proto` under a user-supplied folder, emit a self-contained `.resource` file with typed keywords (one per RPC method) that wrap QConnectBase's `GrpcClient` connection.

Test authors then write:

> *** Settings*** Resource com_setup_device_service.resource
>
> *** Test Cases*** Smoke Com Setup Device Service Connect conn=power ... service_name=multi_proto consul_addr=http://127.0.0.1:8500 ... proto_dir=${CURDIR}/../proto ${res}= Com Setup Device Service Set Interface Type conn=power type=0 Should Be Equal As Integers ${res}[result] 0 [Teardown] Com Setup Device Service Disconnect power

. . . rather than hand-rolling JSON `send_cmd` strings.

Entry points:

- `generate_robot_resources` — programmatic API; returns `{relative_path: file_content}`.
- `MicroserviceBase.tools.robot_gen` — CLI wrapper.
- `POST /api/scaffold/robot` — FastAPI endpoint used by the GUI.

62.1 Function: parse_proto_folder

Walk `proto_dir` for `.proto` files and return discovered services.

Each `.proto` is compiled via `grpc_tools.protoc` into a FileDescriptorSet; descriptors carry the service / method / message information we need to emit typed keywords.

Only the top-level proto folder is scanned non-recursively. Imports that reference other `.proto` files in the same folder are resolved (the folder is passed as a single `-I` include path). microservicebase-adapters-scaffold-robot-tmpl-generate-robot-resources ==========

Generate one `.resource` per service under `proto_dir`.

Returns `{relative_path: file_content}` — the caller writes them.

service_filter (optional): only emit resources for services whose name is in the list. Empty / None = all. microservicebase-adapters-scaffold-robot-tmpl--rpcparam ==========

Imported by:

```
from MicroserviceBase.adapters.scaffold.robot_tmpl import _RpcParam
```

62.2 Class: _Rpc

Imported by:

```
from MicroserviceBase.adapters.scaffold.robot_tmpl import _Rpc
```

62.3 Class: _Service

Imported by:

```
from MicroserviceBase.adapters.scaffold.robot_tmpl import _Service
```

62.3.1 Method: full_name

62.4 Class: RobotGenError

Imported by:

```
from MicroserviceBase.adapters.scaffold.robot_tmpl import RobotGenError
```

Raised when proto parsing or emit fails.

Chapter 63

shared.py

Shared scaffold generators used by both Python and C++ templates.

Produces: - .proto file - service_config.json - Nomad .nomad.hcl job spec - README.md

63.1 Function: python_file_header

Render the standard QConnect-style Python file header block.

Format mirrors the upstream QConnectBase convention:

- Apache 2.0 boilerplate
- File / Author / Date / Description / History sections
- Author taken from `_full_username` (full PC user name)
- Date and history entry generated from today's date

Arguments:

- `filename`
  / *Condition*: required / *Type*: str /
  Bare filename (no path), e.g. `"main.py"`.
- `description`
  / *Condition*: required / *Type*: str /
  One-paragraph description of what the module provides. May contain newlines; will be re-indented to fit the comment column.
- `version`
  / *Condition*: optional / *Type*: str / *Default*: "1.0.0" /
  Initial version string written in the History entry.

Returns:

- `header`
  / *Type*: str /
  Multi-line #-comment header, ending with a blank line for clean separation from the module docstring that follows.

63.2 Function: cpp_file_header

Render the QConnect-style file header as a C/C++ `//` comment block.

Same content as `python_file_header` but with `//` line comments so it compiles inside a `.h` / `.cpp` source file.

Arguments:

- `filename`
  / *Condition*: required / *Type*: str /
  Bare filename, e.g. `"main.cpp"` or `"HelloService.h"`.
- `description`
  / *Condition*: required / *Type*: str /
  Module / file description. Multi-line allowed.
- `version`
  / *Condition*: optional / *Type*: str / *Default*: "1.0.0" /
  Initial version string.

Returns:

- `header`
  / *Type*: str /
  Multi-line `//`-comment header, ending with a blank line.

63.3 Function: gen_proto

Render the `.proto` file for a single service.

Arguments:

- `spec`
  / *Condition*: required / *Type*: ScaffoldSpec /
  Scaffold specification (service name, methods, package, ...).

Returns:

- `proto`
  / *Type*: str /
  The full `.proto` text, including the service block and one request/response message per method.

63.4 Function: gen_service_config

Render the `service_config.json` metadata file for a service.

The shape carries broker-era fields (`broker_*`) for back-compat with the legacy Manager GUI's service-info reader. New gRPC-only deployments ignore those fields.

Arguments:

- `spec`
  / *Condition*: required / *Type*: ScaffoldSpec /
  Scaffold specification.

Returns:

- `json_text`
  / *Type*: str /
  Pretty-printed JSON document (4-space indent, trailing newline).

63.5 Function: gen_nomad

Render the <service>.nomad.hcl job spec.

Wires the dynamic NOMAD_PORT_grpc allocation into <PREFIX>_GRPC_PORT and <PREFIX>_ADVERTISE_ADDR env vars; sets <PREFIX>_CONSUL_ADDR to the configured Consul address so ServiceRunner can register the gRPC port automatically.

Arguments:

- spec
  / *Condition*: required / *Type*: ScaffoldSpec /
  Scaffold specification. Reads nomad_dc, nomad_driver, nomad_cpu, nomad_mem, nomad_consul_addr if present.

Returns:

- hcl
  / *Type*: str /
  Nomad HCL job definition, ready to nomad job run.

63.6 Function: gen_readme

Render the README.md for a generated service.

Output covers: project layout, build instructions for the language / GUI / toolchain combination the wizard picked, run instructions (direct + Nomad), and the list of RPC methods exposed by the service.

Arguments:

- spec
  / *Condition*: required / *Type*: ScaffoldSpec /
  Scaffold specification.

Returns:

- markdown
  / *Type*: str /
  README contents (Markdown).

Chapter 64

eventbus_adapter.py

64.1 Class: _ChannelProxy

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _ChannelProxy
```

Lightweight proxy that mimics a pika channel for the domain handler.
Translates basic_publish and basic_ack calls into EventBusClient operations.

64.1.1 Method: basic_publish

Publish a reply message via the EventBusClient.
Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name (ignored, EventBusClient uses its configured exchange).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the reply (the reply_to value).
- `properties`
  / *Condition*: optional / *Type*: object /
  Properties object with correlation_id attribute.
- `body`
  / *Condition*: optional / *Type*: str or bytes /
  The message body (JSON string or bytes).

64.1.2 Method: basic_ack

Acknowledge a message (no-op for EventBusClient which auto-acks).

64.2 Class: _MethodProxy

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _MethodProxy
```

Lightweight proxy that mimics a pika method frame.

64.3 Class: _PropertiesProxy

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import _PropertiesProxy
```

Lightweight proxy that mimics pika BasicProperties.

64.4 Class: EventBusTransportAdapter

Imported by:

```
from MicroserviceBase.adapters.transport.eventbus_adapter import
↪  EventBusTransportAdapter
```

EventBusClient implementation of the TransportPort interface.

Creates the underlying EventBusClient from a JSONP configuration file via `EventBusClient.from_config_sync()`.

64.4.1 Method: connect

Create the EventBusClient from the JSONP config and establish connection.

64.4.2 Method: disconnect

Close the EventBusClient connection.

64.4.3 Method: consume

Start consuming RPC requests for a service.

Subscribes to the routing key and blocks the calling thread. Incoming messages are adapted to the pika-style handler signature.

Arguments:

- `service_name`
  / *Condition*: required / *Type*: str /
  Name used as the service identifier.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for message binding.
- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name (must match the configured exchange).
- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, props, body).

64.4.4 Method: rpc_call

Send an RPC request and wait for the response.

Creates a temporary subscription for the reply, sends the request with reply_to and correlation_id headers, and waits for the response.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  dict to send as the request body.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  Exchange to publish to (must match configured exchange).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the target service.

Returns:

Parsed response dict.

64.4.5 Method: publish

Publish a message to the exchange.

Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name (must match configured exchange).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key.
- `body`
  / *Condition*: required / *Type*: str, bytes, or dict /
  Message body.
- `properties`
  / *Condition*: optional / *Type*: dict /
  Optional message properties (used for headers).

Chapter 65

rabbitmq_adapter.py

65.1 Class: RabbitMQTransportAdapter

Imported by:

```
from MicroserviceBase.adapters.transport.rabbitmq_adapter import
↪  RabbitMQTransportAdapter
```

RabbitMQ implementation of the TransportPort interface.

65.1.1 Method: connect

Establish connection to RabbitMQ.

65.1.2 Method: disconnect

Close connection to RabbitMQ.

65.1.3 Method: connection

Access the underlying pika connection (for backward compat).

65.1.4 Method: stop_consuming

Signal the consume loop to exit.

65.1.5 Method: consume

Start consuming RPC requests for a service.

Sets up the exchange, queue, binding, and starts blocking consumption.

Arguments:

- `service_name`
  / *Condition*: required / *Type*: str /
  Name used as the queue name.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for queue binding.

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name to bind to.
- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, props, body).

65.1.6 Method: rpc_call

Send an RPC request and wait for the response.
Creates a new connection for the RPC call (matching original behavior).
Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  dict to serialize as the request body.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  Exchange to publish to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the target service.
- `timeout`
  / *Condition*: optional / *Type*: int / *Default*: 30 /
  Timeout in seconds for waiting for the response.

Returns:

Parsed response dict.

65.1.7 Method: publish

Publish a message to an exchange.
Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key.
- `body`
  / *Condition*: required / *Type*: str or bytes /
  Message body (str or bytes).
- `properties`
  / *Condition*: optional / *Type*: pika.BasicProperties /
  Optional pika.BasicProperties.

Chapter 66

fastapi_bridge.py

66.1 Class: FastAPIBridge

Imported by:

```
from MicroserviceBase.adapters.ui_bridge.fastapi_bridge import FastAPIBridge
```

FastAPI implementation of UIBridgePort.

Exposes a REST API that any UI client (Electron, browser, mobile) can call. Also provides a WebSocket endpoint for push-based service updates.

Endpoints: POST /api/request - Route a service request through the transport GET /api/services - Get current services info WS /ws/updates - Real-time service update stream

Requires `fastapi` and `uvicorn` (optional dependencies).

66.1.1 Method: start

Start the FastAPI server in a background thread.

Arguments:

- `request_handler`
  / *Condition*: required / *Type*: callable /
  Callable(request_data, exchange, routing_key) -> response dict.
- `services_info_provider`
  / *Condition*: required / *Type*: callable /
  Callable() -> services info dict.

66.1.2 Method: wait

Block the calling thread until the server thread exits.

Uses a loop with timeout so that KeyboardInterrupt can be caught on Windows.

66.1.3 Method: stop

Stop the FastAPI server.

66.1.4 Method: broadcast_update

Push a services update to all connected WebSocket clients.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  dict of all current service information.

Chapter 67

exceptions.py

67.1 Class: ServiceError

Imported by:

```
from MicroserviceBase.domain.exceptions import ServiceError
```

Base exception for service-related errors.

67.2 Class: TransportError

Imported by:

```
from MicroserviceBase.domain.exceptions import TransportError
```

Raised when a transport-level operation fails.

67.3 Class: ServiceNotFoundError

Imported by:

```
from MicroserviceBase.domain.exceptions import ServiceNotFoundError
```

Raised when a requested service is not found in the registry.

67.4 Class: MethodNotFoundError

Imported by:

```
from MicroserviceBase.domain.exceptions import MethodNotFoundError
```

Raised when a requested method is not found on a service.

Chapter 68

messages.py

68.1 Class: ResultType

Imported by:

```
from MicroserviceBase.domain.messages import ResultType
```

Result types for service responses.

68.2 Class: ServiceRequest

Imported by:

```
from MicroserviceBase.domain.messages import ServiceRequest
```

Structured request to a service method.

68.2.1 Method: to_dict

Converts this `ServiceRequest` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this request.

68.2.2 Method: to_json

Converts this `ServiceRequest` instance to a JSON string.

Returns:

/ *Type*: str /
JSON string representation of this request.

68.2.3 Method: from_dict

Creates a `ServiceRequest` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing request fields.

Returns:

/ *Type*: ServiceRequest /
New instance populated from the dictionary.

68.2.4 Method: from_json

Creates a `ServiceRequest` instance from a JSON string.

Arguments:

- `json_str`
  / *Condition*: required / *Type*: str /
  JSON string containing request fields.

Returns:

/ *Type*: ServiceRequest /
New instance populated from the JSON string.

68.3 Class: ServiceResponse

Imported by:

```
from MicroserviceBase.domain.messages import ServiceResponse
```

Structured response from a service method.

68.3.1 Method: to_dict

Converts this `ServiceResponse` instance to an ordered dictionary.

Returns:

/ *Type*: OrderedDict /
Sorted ordered dictionary representation of this response.

68.3.2 Method: to_json

Converts this `ServiceResponse` instance to a JSON string.

Returns:

/ *Type*: str /
JSON string representation of this response.

68.3.3 Method: get_json

Backward-compatible alias for `to_json()`.

Returns:

/ *Type*: str /
JSON string representation of this response.

68.3.4 Method: get_data

Backward-compatible alias for `result_data`.

Returns:

/ *Type*: Any /
The result data of this response.

68.3.5 Method: from_dict

Creates a `ServiceResponse` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing response fields.

Returns:

/ *Type*: ServiceResponse /
New instance populated from the dictionary.

68.3.6 Method: from_json

Creates a `ServiceResponse` instance from a JSON string.

Arguments:

- `json_str`
  / *Condition*: required / *Type*: str /
  JSON string containing response fields.

Returns:

/ *Type*: ServiceResponse /
New instance populated from the JSON string.

68.4 Class: ServiceEvent

Imported by:

```
from MicroserviceBase.domain.messages import ServiceEvent
```

Event emitted when service state changes.

68.4.1 Method: to_dict

Converts this `ServiceEvent` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this event.

68.4.2 Method: to_json

Converts this `ServiceEvent` instance to a JSON string.

Returns:

/ *Type*: str /
JSON string representation of this event.

68.4.3 Method: from_dict

Creates a `ServiceEvent` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing event fields.

Returns:

/ *Type*: ServiceEvent /
New instance populated from the dictionary.

68.4.4 Method: from_json

Creates a `ServiceEvent` instance from a JSON string.

Arguments:

- `json_str`
  / *Condition*: required / *Type*: str /
  JSON string containing event fields.

Returns:

/ *Type*: ServiceEvent /
New instance populated from the JSON string.

Chapter 69

models.py

69.1 Class: MethodArgument

Imported by:

```
from MicroserviceBase.domain.models import MethodArgument
```

Describes a single argument of a service API method.

69.1.1 Method: to_dict

Converts this `MethodArgument` instance to a dictionary.
Returns:

/ *Type*: dict /
Dictionary representation of this argument.

69.1.2 Method: from_dict

Creates a `MethodArgument` instance from a dictionary.
Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing argument fields.

Returns:

/ *Type*: MethodArgument /
New instance populated from the dictionary.

69.2 Class: ServiceMethod

Imported by:

```
from MicroserviceBase.domain.models import ServiceMethod
```

Describes a single API method exposed by a service.

69.2.1 Method: to_dict

Converts this `ServiceMethod` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this method.

69.2.2 Method: from_dict

Creates a `ServiceMethod` instance from a name and dictionary.

Arguments:

- `name`
  / *Condition*: required / *Type*: str /
  The method name.
- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing method fields.

Returns:

/ *Type*: ServiceMethod /
New instance populated from the dictionary.

69.3 Class: ServiceInfo

Imported by:

```
from MicroserviceBase.domain.models import ServiceInfo
```

Describes a service and its metadata.

69.3.1 Method: to_dict

Converts this `ServiceInfo` instance to a dictionary.

Returns:

/ *Type*: dict /
Dictionary representation of this service info.

69.3.2 Method: from_dict

Creates a `ServiceInfo` instance from a dictionary.

Arguments:

- `data`
  / *Condition*: required / *Type*: dict /
  Dictionary containing service info fields.

Returns:

/ *Type*: ServiceInfo /
New instance populated from the dictionary.

Chapter 70

service_base.py

70.1 Class: ServiceBase

Imported by:

```
from MicroserviceBase.domain.service_base import ServiceBase
```

Pure domain base class for services.

All methods used to export APIs of a service must begin with the prefix 'svc_api'. This class handles API discovery, docstring parsing, and request dispatch. Transport and registry interactions are delegated to injected ports.

70.1.1 Method: get_service_info

Return the ServiceInfo dataclass for this service.

70.1.2 Method: get_service_info_dict

Return the service info as a plain dict (backward compat).

70.1.3 Method: serve

Start serving requests via the transport port.

70.1.4 Method: register_service

Register this service via the registry port.

70.1.5 Method: unregister_service

Unregister this service via the registry port.

70.1.6 Method: request_service

Send an RPC request to another service via the transport port.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  Dict with 'method' and 'args' keys (backward compat format).

- `exchange_name`
  / *Condition*: required / *Type*: str /
  The exchange to publish to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  The routing key for the target service.

Returns:

/ *Type*: dict /
The response dict from the target service.

70.1.7 Method: close

Close the service.

70.1.8 Method: create_request_data

Create request data for a given method name and arguments.

70.1.9 Method: get_svc_api_methods_dict

Retrieve all service API methods (methods starting with 'svc_api').

70.1.10 Method: get_svc_api_methods_info_dict

Retrieve information for all service APIs from their docstrings.

70.1.11 Method: parse_docstring

Parse function's docstring to get arguments and return information.

70.1.12 Method: svc_api_get_version

Get the service version.

Returns:

/ *Type*: str /
Version of the service.

70.1.13 Method: svc_api_get_gui_files

Compress and return GUI files as bytes.

70.1.14 Method: svc_api_get_service_files

Compress and return the entire service directory as bytes.

Returns:

/ *Type*: bytes /
ZIP archive of the service directory, or None if not downloadable.

70.1.15 Method: svc_api_get_gui_checksum

Get an MD5 checksum of all GUI files.

Returns None when gui_support is disabled or the GUIs directory is missing.

Returns:

/ *Type*: str /
MD5 hex-digest of the GUI files, or None.

70.1.16 Method: svc_api_shutdown

Gracefully shut down this service.

70.1.17 Method: is_specific_request

Check if the request is a specific request. Override in subclasses.

70.1.18 Method: on_specific_request

Handle a specific request. Override in subclasses.

Arguments:

- `request`
  / *Condition*: required / *Type*: str /
  The request method name.
- `body`
  / *Condition*: required / *Type*: dict /
  The parsed request body dict.

Returns:

/ *Type*: ServiceResponse /
A ServiceResponse, or None if not handled.

70.1.19 Method: dispatch_request

Dispatch a parsed request body and return a ServiceResponse.

This is the core domain logic: given a request dict with 'method' and 'args', find the matching API method, call it, and return a structured response.

Arguments:

- `request_body`
  / *Condition*: required / *Type*: dict /
  Dict with 'method' and 'args' keys.

Returns:

/ *Type*: ServiceResponse /
ServiceResponse with result type and data.

70.1.20 Method: on_request

Handle an incoming request (transport callback signature).

This method is called by the transport adapter. It delegates to dispatch_request for the actual domain logic, then uses the transport to send the response back.

Arguments:

- `ch`
  / *Condition*: required / *Type*: object /
  Channel object from transport.
- `method`
  / *Condition*: required / *Type*: object /
  Delivery method from transport.
- `props`
  / *Condition*: required / *Type*: object /
  Message properties from transport.
- `body`
  / *Condition*: required / *Type*: bytes /
  Raw message body (bytes or dict).

Chapter 71

service_registry.py

71.1 Class: ServiceRegistry

Imported by:

```
from MicroserviceBase.domain.service_registry import ServiceRegistry
```

Pure domain registry that tracks services, handles aliases, and routes requests.

71.1.1 Method: handle_update

Handle a service registration/unregistration event.

Arguments:

- `service_information`
  / *Condition*: required / *Type*: dict /
  Dict with 'info' and 'state' keys.

71.1.2 Method: notify_updates

Notify connected clients about service updates via the registry port.

71.1.3 Method: svc_api_shutdown

Gracefully shut down the Service Registry.

71.1.4 Method: svc_api_get_services_info

Retrieve information of all services connected to the broker.

Returns:

/ *Type*: dict /
A dictionary containing information of all connected services.

71.1.5 Method: svc_api_get_realtime_update_exchange

Retrieve the exchange name of the realtime update exchange.

Returns:

/ *Type*: str /
The name of the realtime update exchange.

71.1.6 Method: svc_api_update_alias_conf

Update the alias configuration information.

Arguments:

- `alias_string`
  / *Condition*: required / *Type*: str /
  The alias configuration string to be updated.

Returns:

(*no returns*)

71.1.7 Method: svc_api_get_alias_conf

Retrieve the alias configuration string in JSON format.

Returns:

/ *Type*: str /
The alias configuration string in JSON format.

71.1.8 Method: is_specific_request

Check if the request is an alias-routed request.

71.1.9 Method: on_specific_request

Handle an alias-routed request by forwarding to the target service.

71.1.10 Method: handle_alias_request

Handle an alias request by routing to the actual service.

Arguments:

- `body`
  / *Condition*: required / *Type*: dict /
  Dict with 'method' and 'args' keys.

Returns:

/ *Type*: ServiceResponse /
ServiceResponse or raw response dict from the target service.

71.1.11 Method: run_health_check_loop

Blocking loop for a daemon thread. Periodically checks whether each registered service still has active RabbitMQ consumers.

Arguments:

- `interval`
  / *Condition*: optional / *Type*: int / *Default*: 30 /
  Seconds between health-check cycles.
- `max_failures`
  / *Condition*: optional / *Type*: int / *Default*: 2 /
  Consecutive zero-consumer checks before auto-unregister.

- `broker_host`
  / *Condition*: optional / *Type*: str / *Default*: 'localhost' /
  RabbitMQ broker host for the temporary connection.
- `broker_port`
  / *Condition*: optional / *Type*: int / *Default*: 5672 /
  RabbitMQ broker port for the temporary connection.

71.1.12 Method: stop_health_check

Signal the health-check loop to stop.

Chapter 72

factory.py

72.1 Function: create_transport

Create a TransportPort implementation.

Arguments:

- `transport_type`
  / *Condition*: optional / *Type*: str / *Default*: 'rabbitmq' /
  Transport backend identifier. Supports 'rabbitmq' and 'eventbus'.
- `cmd_args`
  / *Condition*: optional / *Type*: list /
  Command-line arguments for config parsing (used by 'rabbitmq').
- `service_name`
  / *Condition*: optional / *Type*: str / *Default*: 'Service' /
  Service name for help text (used by 'rabbitmq').
- `config_path`
  / *Condition*: optional / *Type*: str /
  Path to JSONP config file (used by 'eventbus').

Returns:

/ *Type*: TransportPort /
A connected TransportPort instance.

72.2 Function: create_registry

Create a ServiceRegistryPort implementation.

Arguments:

- `transport_type`
  / *Condition*: optional / *Type*: str / *Default*: 'rabbitmq' /
  Transport backend identifier. Supports 'rabbitmq' and 'eventbus'.
- `cmd_args`
  / *Condition*: optional / *Type*: list /
  Command-line arguments for config parsing (used by 'rabbitmq').

- `service_name`
  / *Condition*: optional / *Type*: str / *Default*: 'Service' /
  Service name for help text (used by 'rabbitmq').
- `update_exchange_name`
  / *Condition*: optional / *Type*: str /
  Name for the realtime update fanout exchange.
- `config_path`
  / *Condition*: optional / *Type*: str /
  Path to JSONP config file (used by 'eventbus').

Returns:

/ *Type*: ServiceRegistryPort /
A ServiceRegistryPort instance.

72.3 Function: create_ui_bridge

Create a UIBridgePort implementation.

Arguments:

- `bridge_type`
  / *Condition*: optional / *Type*: str / *Default*: 'fastapi' /
  Bridge backend identifier. Currently supports 'fastapi'.
- `host`
  / *Condition*: optional / *Type*: str / *Default*: 'localhost' /
  Host to bind to.
- `port`
  / *Condition*: optional / *Type*: int / *Default*: 8000 /
  Port to bind to.

Returns:

/ *Type*: UIBridgePort /
A UIBridgePort instance (not yet started).

Chapter 73

hub_manager.py

73.1 Class: HubManagerPort

Imported by:

```
from MicroserviceBase.ports.hub_manager import HubManagerPort
```

Abstract interface for process/job orchestration backends.

Implementations manage service lifecycles through different backends: - LocalHubManager: local OS processes via ProcessHub - NomadHubAdapter: Nomad jobs via HTTP API

73.1.1 Method: start_hub

Start the orchestration backend.

Arguments:

- `mode`
  / *Condition*: optional / *Type*: str / *Default*: 'standalone' /
  Operating mode. Backend-specific values.

Returns:

- `dict` with at least {`"status": str`}

73.1.2 Method: stop_hub

Stop the orchestration backend and release resources.

Returns:

- `dict` with at least {`"status": str`}

73.1.3 Method: get_status

Get a status snapshot of all managed services/jobs.

Returns:

- `dict` containing `running`, `mode`, `processes` list, etc.

73.1.4 Method: start_processes

Start named services/jobs.

Arguments:

- `names`
  / *Condition*: required / *Type*: list[str] /
  List of service/process names to start.

Returns:

- `dict` mapping name to `{"success": bool, "message": str}`

73.1.5 Method: stop_processes

Stop named services/jobs.
Arguments:

- `names`
  / *Condition*: required / *Type*: list[str] /
  List of service/process names to stop.
- `force`
  / *Condition*: optional / *Type*: bool / *Default*: False /
  Skip graceful shutdown and force-kill immediately.

Returns:

- `dict` mapping name to `{"success": bool, "message": str}`

73.1.6 Method: get_config

Get all service/job configurations.
Returns:

- `dict` of process/job configs keyed by name.

73.1.7 Method: add_config

Add a new service/job configuration.
Arguments:

- `name`
  / *Condition*: required / *Type*: str /
- `config`
  / *Condition*: required / *Type*: dict /

Returns:

- `dict` with `{"success": bool, "message": str}`

73.1.8 Method: update_config

Update an existing service/job configuration.
Returns:

- `dict` with `{"success": bool, "message": str}`

73.1.9 Method: remove_config

Remove a service/job configuration.

Returns:

- `dict` with `{"success": bool, "message": str}`

73.1.10 Method: get_service_log

Read service/job log output.

Returns:

- `dict` with `{"name": str, "log": str}`

73.1.11 Method: import_service

Import a service package.

Returns:

- `dict` with `{"success": bool, "message": str}`

73.1.12 Method: remove_service

Remove a service entirely (stop + delete config + files).

Returns:

- `dict` with `{"success": bool, "message": str}`

73.1.13 Method: reset

Reset the hub — stop all services and clear state.

Returns:

- `dict` with `{"success": bool, "message": str}`

Chapter 74

registry.py

74.1 Class: ServiceRegistryPort

Imported by:

```
from MicroserviceBase.ports.registry import ServiceRegistryPort
```

Abstract interface for service registration and discovery.
Implementations handle how services announce themselves and how the registry discovers/tracks them.

74.1.1 Method: register

Register a service with the registry.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  Dict containing service metadata.

74.1.2 Method: unregister

Unregister a service from the registry.
Arguments:

- `service_info`
  / *Condition*: required / *Type*: dict /
  Dict containing service metadata.

74.1.3 Method: subscribe_to_events

Subscribe to service registration/unregistration events.
Arguments:

- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, properties, body) for events.

74.1.4 Method: notify_update

Broadcast a services update to all subscribers.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  Dict of all current service information.

74.1.5 Method: get_update_channel_name

Return the name of the realtime update channel/exchange.

Returns:

Channel/exchange name as a string.

Chapter 75

transport.py

75.1 Class: TransportPort

Imported by:

```
from MicroserviceBase.ports.transport import TransportPort
```

Abstract interface for message transport.

Implementations provide the actual connectivity (e.g., RabbitMQ, Redis, MQTT). The domain layer depends only on this interface.

75.1.1 Method: connect

Establish connection to the message broker.

75.1.2 Method: disconnect

Close connection to the message broker.

75.1.3 Method: stop_consuming

Signal the consume loop to stop.

75.1.4 Method: consume

Start consuming messages for a service.

Arguments:

- `service_name`
  / *Condition*: required / *Type*: str /
  Name of the service (used as queue name).
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key to bind the queue.
- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name to bind to.
- `handler`
  / *Condition*: required / *Type*: callable /
  Callback function(ch, method, props, body) for incoming messages.

75.1.5 Method: rpc_call

Send an RPC request and wait for the response.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  Dict to serialize and send as the request body.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  Exchange to publish to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key for the target service.

Returns:

Parsed response dict from the target service.

75.1.6 Method: publish

Publish a message to an exchange.

Arguments:

- `exchange`
  / *Condition*: required / *Type*: str /
  Exchange name.
- `routing_key`
  / *Condition*: required / *Type*: str /
  Routing key.
- `body`
  / *Condition*: required / *Type*: str or bytes /
  Message body.
- `properties`
  / *Condition*: optional / *Type*: dict /
  Optional message properties dict.

Chapter 76

ui_bridge.py

76.1 Class: UIBridgePort

Imported by:

```
from MicroserviceBase.ports.ui_bridge import UIBridgePort
```

Abstract interface for UI communication bridge.

Implementations provide the actual UI connectivity (e.g., FastAPI REST, gRPC). Any frontend (Electron, browser, mobile) can connect through this port.

76.1.1 Method: start

Start the UI bridge server.

Arguments:

- `request_handler`
  / *Condition*: required / *Type*: callable /
  Callable that accepts a ServiceRequest dict and returns a ServiceResponse dict. Used to route UI requests to services.
- `services_info_provider`
  / *Condition*: required / *Type*: callable /
  Callable that returns the current services info dict.

76.1.2 Method: stop

Stop the UI bridge server.

76.1.3 Method: broadcast_update

Push a services update to all connected UI clients.

Arguments:

- `services_info`
  / *Condition*: required / *Type*: dict /
  Dict of all current service information.

Chapter 77

__init__.py

MicroserviceBase runtime library.
Public API used by service authors:

- `ServiceRunner` — start a gRPC service with Consul registration, reflection, and graceful shutdown.
- `ServicerEntry` — one servicer to bind to the gRPC server.
- `ConsulRegistration` — low-level Consul register/deregister.
- `BaseServiceSettings` — Pydantic parent class for service settings.

Chapter 78

consul.py

Consul service registration and lookup.

Direct HTTP client (no python-consul dependency). Consul's HTTP API is small and stable; adding a third-party wrapper adds a dependency without value.

The `ConsulRegistration` class is used as an async context manager:

```
async with ConsulRegistration(
    name="hello",
    address="127.0.0.1",
    port=50051,
    consul_addr="http://localhost:8500",
):
    await server.wait_for_termination()
```

The service is registered when the context is entered and deregistered on exit. A gRPC health check is registered against the same port so Consul can mark the service healthy / unhealthy automatically.

For client-side lookup, `resolve_via_consul` returns the first healthy `host:port` registered under a service name.

78.1 Function: resolve_via_consul

Resolve a Consul service name to a `host:port` string.

Calls `GET /v1/health/service/<name>?passing=true` and returns the first healthy instance. Synchronous (uses `httpx.Client`) — designed for client-side bootstrap of a single channel, not for hot paths.

For per-call resolution, prefer the gRPC channel's built-in `consul://` resolver in MicroserviceBase's C++ `ServiceClient<T>`, or use the bridge's `/api/consul/health/<svc>` endpoint.

Arguments:

- `consul_addr`
  / *Condition*: required / *Type*: str /
  Consul HTTP API endpoint, e.g. `"http://localhost:8500"`.
- `service_name`
  / *Condition*: required / *Type*: str /
  Logical service name as registered with Consul.
- `timeout`
  / *Condition*: optional / *Type*: float / *Default*: 5.0 /
  HTTP request timeout in seconds.

Returns:

- `endpoint`
  / *Type*: str /
  `host:port` string of the first healthy instance. Empty string when no healthy instance is registered.

Raises:

- `httpx.HTTPError`
  When the Consul API call itself fails (network error, 5xx, etc.).

78.2 Class: ConsulRegistration

Imported by:

```
from MicroserviceBase.runtime.consul import ConsulRegistration
```

Async context manager that registers a service with Consul on entry and deregisters it on exit.

The service ID is generated from the service name and a short random suffix so multiple instances of the same service can coexist on the same node.

78.2.1 Method: service_id

The randomly-suffixed service ID this registration uses.

Returns:

- `service_id`
  / *Type*: str /
  Concatenation of service name and a short hex suffix.

Chapter 79

reflection.py

gRPC reflection helper.

Enables the standard gRPC server reflection protocol so clients can list services and methods at runtime — which is how `MicroserviceManagerGUI` discovers the method surface of each registered service without needing to know each `.proto` ahead of time.

79.1 Function: enable_reflection

Register the reflection service on *server*.

Args: server: The running async gRPC server. service_names: Fully-qualified service names to advertise, e.g. `["hello.v1.HelloService", ...]`. The reflection service itself is added automatically.

Chapter 80

server.py

Service runtime.

`ServiceRunner` is the single entry point a service author uses to:

- Start an async gRPC server
- Bind a configurable list of servicers
- Enable gRPC reflection so clients can discover methods
- Enable the standard gRPC health check protocol
- Register the service with Consul using the (possibly OS-assigned) port
- Deregister + shut down gracefully on SIGINT/SIGTERM/SIGBREAK

Usage (simplified):

```
runner = ServiceRunner(
    service_name="hello",
    servicers=[(hello_pb2_grpc.add_HelloServiceServicer_to_server,
                hello_pb2.DESCRIPTOR.services_by_name["HelloService"].full_name,
                HelloGrpcAdapter(domain))],
    settings=settings,
)
await runner.serve_forever()
```

The `servicers` argument is a list of tuples: `(register_fn, full_service_name, servicer_instance)`.

80.1 Class: ServicerEntry

Imported by:

```
from MicroserviceBase.runtime.server import ServicerEntry
```

One gRPC servicer to bind to the server.

Attributes:

- `register_fn`
  / *Type*: callable /
  The `add_<Name>Servicer_to_server` function generated by `grpcio_tools`.
- `full_service_name`
  / *Type*: str /
  Fully-qualified proto service name (e.g. `"hello.v1.HelloService"`). Used for reflection + optional Consul metadata.

- `servicer`
  / *Type*: object /
  The servicer instance implementing the RPCs.

80.2 Class: ServiceRunner

Imported by:

```
from MicroserviceBase.runtime.server import ServiceRunner
```

Run an async gRPC service with Consul registration and graceful shutdown.
`settings.service_name` must be non-empty; it's the logical name other services and the GUI use to find this one.

80.2.1 Method: request_shutdown

Trigger graceful shutdown. Safe to call from a signal handler.
Returns:
(*no returns*)

Chapter 81

settings.py

Base Pydantic settings for microservices.

Every service subclasses `BaseServiceSettings` and adds its own fields. Environment variables are loaded with a service-specific prefix.

Example:

```
class HelloSettings(BaseServiceSettings):
    model_config = SettingsConfigDict(env_prefix="HELLO_", env_file=".env")

    greeting: str = "Hello"
```

The base provides the fields that every service needs to talk to Consul and expose a gRPC server.

81.1 Class: BaseServiceSettings

Imported by:

```
from MicroserviceBase.runtime.settings import BaseServiceSettings
```

Common settings shared by all microservices.

Subclasses add service-specific fields and set `env_prefix` to the service's name (e.g. `HELLO_`).

Attributes:

- `service_name`

  / *Type*: str / *Default*: "" /

  Logical service name used for Consul registration.

- `service_host`

  / *Type*: str / *Default*: "0.0.0.0" /

  Address the gRPC server binds to. `0.0.0.0` makes the service reachable on any interface; Consul registers the advertised address from `advertise_addr` if set, otherwise falls back to the local hostname.

- `grpc_port`

  / *Type*: int / *Default*: 0 /

  TCP port for the gRPC server. `0` means "let the operating system pick a free port". The chosen port is then announced to Consul.

- `advertise_addr`

  / *Type*: str / *Default*: "" /

  Address announced to Consul. Leave empty to use the local hostname.

- `consul_addr`
  / *Type*: str / *Default*: "`http://localhost:8500`" /
  Consul HTTP API endpoint.
- `consul_token`
  / *Type*: str / *Default*: "" /
  Optional ACL token for Consul.
- `log_level`
  / *Type*: str / *Default*: "INFO" /
  Python logging level name (`DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`).

Chapter 82

robot_gen.py

Generate Robot Framework resource files from a .proto folder.

For every `service` declared in any `.proto` under `--proto-dir`, emit a self-contained `<snake_service>.resource` file with one keyword per RPC method. See `MicroserviceBase.adapters.scaffold.robot_tmpl` for the underlying generator.

Standalone — does NOT require the bridge to be running. Calls the generator in-process so it works in CI / from a plain shell.

Usage:

```
python -m MicroserviceBase.tools.robot_gen \
    --proto-dir D:/Project/.../MultiProto2/proto \
    --out      ./tests/resources

# Only generate one of the services
python -m MicroserviceBase.tools.robot_gen \
    --proto-dir ./proto --out ./tests \
    --service ComSetupDeviceService

# Print to stdout instead of writing files (handy in pipes)
python -m MicroserviceBase.tools.robot_gen --proto-dir ./proto --stdout
```

82.1 Function: main

Chapter 83

scaffold_cli.py

Generate microservice scaffolds via the bridge's HTTP API.

Does the same job as the Service Creator wizard in the Manager GUI, but from a terminal — so you can script it, commit reproducible specs, or run from CI.

Two ways to supply input:

1. **Config file** (YAML or JSON) with any subset of the `/api/scaffold/generate-v2` fields:

   ```
   python -m MicroserviceBase.tools.scaffold_cli --config my_service.yaml
   ```

2. **Inline flags** for the common cases:

   ```
   python -m MicroserviceBase.tools.scaffold_cli \
       --name Hello --language python --output ./out
   ```

Config + flags compose: flags override fields loaded from the config.

Importing an existing .proto is supported via `--proto path/to/file.proto`. Pass `--monorepo` to emit one project folder with N executables (one per service declared in the .proto).

Requires the bridge to be running (default `http://127.0.0.1:1112`; set via `--bridge-url` or the `MB_BRIDGE_URL` env var).

83.1 Function: main

Chapter 84

Appendix

84.1 Appendix

84.1.1 License

MicroserviceBase is licensed under the Apache License, Version 2.0.
You may obtain a copy of the License at: http://www.apache.org/licenses/LICENSE-2.0

84.1.2 References

- RabbitMQ: https://www.rabbitmq.com
- pika (RabbitMQ client for Python): https://pika.readthedocs.io
- FastAPI: https://fastapi.tiangolo.com
- Electron: https://www.electronjs.org
- Python: https://www.python.org

84.1.3 Repository

Source code: https://github.com/test-fullautomation/python-microservice-base
Issue tracker: https://github.com/test-fullautomation/python-microservice-base/issues

84.1.4 Contact

Author: Nguyen Huynh Tri Cuong
Email: Cuong.NguyenHuynhTri@vn.bosch.com

Chapter 85

History

85.1 History

85.1.1 Version 2.1.0 - 11.05.2026

First minor release after the gRPC + Consul + Nomad migration shipped in 2.0.0. Covers the Manager-GUI parity work, the Service Creator wizard upgrades, the installer modernisation, and the new Robot Framework test path.

Architecture migration to gRPC + Consul + Nomad

- **Transport**: RabbitMQ (custom request/response framing) replaced by gRPC over HTTP/2 with protobuf payloads.
- **Discovery**: in-process `ServiceRegistry` replaced by the Consul service catalog (`/v1/health/service/...`).
- **Orchestration**: custom Local Process Hub replaced by Nomad jobs (`raw_exec` driver) supervised by a Nomad agent.
- **Routing**: routing keys + alias routing replaced by fully-qualified gRPC service + method names.
- **Topology**: per-service `deploy/<svc>.nomad.hcl` replaces the single `hub_processes.json` config.
- Hexagonal architecture itself is unchanged — the migration swapped adapters, not domain code.

New Architecture Decision Records (ADRs 023–029)

- **ADR-023**: Multi-node Consul cluster with serf gossip.
- **ADR-024**: Multi-node Nomad cluster (server + N clients).
- **ADR-025**: Wrapper layer for Nomad `raw_exec` workloads (PATH setup, env-var derivation, pre-flight checks).
- **ADR-026**: `uv`-based Python service environments (per-service venv, lockfile).
- **ADR-027**: Kafka event bus alongside gRPC (sync = gRPC, fan-out = Kafka).
- **ADR-028**: gRPC + reflection as the canonical RPC layer (consolidates the rationale scattered across the ADR-016/017/018 supersession callouts).
- **ADR-029**: Robot Framework as the primary test client, via the new `GrpcClient` connection type for QConnectBase.
- Older ADRs (003, 010–012, 015–018, 020) carry visible *Superseded* or *Archived* callout banners pointing at the new ones.

Service Creator wizard

- **Multi-proto layout**: one process hosting N gRPC services on the same `ServerBuilder` / `grpc.aio.Server`, one Consul registration, one port. Best for one logical device with multiple capability surfaces.
- **Multi-file `.proto` import**: select N `.proto` files at once; every service from every file gets loaded; layout auto-defaults to `multi_proto`.
- **Three layout options** on the picker modal: `multi_proto`, `monorepo` (one project, N entry points), `separate` (N independent project folders). Inline layout selector also added on Step 3 so users can flip without re-importing.
- **New "GUI Support" wizard step** (Python + HTML/JS only) with two tabs: a Schema Builder that emits `gui_schema.json` (4 component types: Method Form, Result Table, Static Text, Live Status) and a Custom HTML/JS editor with live preview and JS file upload.
- **Server toolchain selector** on Step 2 (MSYS2 prebuilt vs. vcpkg + Qt MinGW) independent of the client gRPC stack choice; mixed configurations get inline warnings.
- **Pre-generate proto stubs** on the bridge runs in-process via `grpc_tools.protoc`; failures now log clearly and the GUI surfaces a yellow toast when stubs were requested but missing.
- **Language-aware copy**: `.exe` references replaced by language-appropriate wording ("Python entry point" for Python, `.exe` for C++).

Generated services: full documentation

- Generated Python and C++ services now ship with QConnect-style file headers (Apache copyright, File / Author / Date / Description / History sections). Author is the OS user's full display name (`GetUserNameExW` on Windows, `pwd.pw_gecos` on Linux); date and history are stamped at generation time.
- Per-method docstrings (Python: QConnect format with **Arguments:** / **Returns:** blocks; C++: Doxygen `@param` / `@return` blocks).
- Class docstrings on Settings, Context, Domain, GrpcAdapter.
- TODO-stub bodies in domain methods describe what the user should document and that the default returns the literal string `"not implemented"`.

Manager GUI

- **Settings dialog**: new *Service infrastructure* section with Consul / Nomad executable path fields and OS file-picker Browse buttons (Electron only). Lookup priority documented inline: this setting → `PATH` → default install location.
- **Service Network mode**: Consul + Nomad dashboards (replaces the legacy Fleet view). Consul tab can start / stop / view-log a managed dev-mode agent; same for Nomad.
- **Bridge endpoints** added: `/api/consul/agent/start + stop + status + log`; same for `/api/nomad/agent/*`. Each persists the spawned agent's PID to a writable data dir for cross-restart reconnection.
- **InfraStatus pills** in the navbar (Consul / Nomad / Bridge) poll every 5 seconds; click jumps to the matching Service Network tab.
- **README + sidebar nav refresh**: documentation map moved from the top of the README to the bottom (less intrusive for first-time readers); per-page sidebar TOCs added.

Installer modernisation (Windows + Linux)

- **NSIS**: new *Infrastructure* wizard page detects Consul / Nomad on `PATH`, in `%ProgramFiles64%\HashiCorp\`, in `%LOCALAPPDATA%\Microsoft\WinGet\Links\`, and in `WinGet\Packages\Hashicorp.*\` (`FindFirst` glob with named labels — `+N` relative jumps and `$PROGRAMDATA` / `$LOCALAPPDATA` replaced with `ReadEnvStr` for portability across NSIS versions).

- **Three install options** per tool: install via winget (pinned versions `Consul 1.20.6` / `Nomad 1.9.6`), provide an existing path, or skip. Winget calls use `--source winget` + `--silent --accept-*-agreements --disable-interactivity`; re-detect-on-failure handles "already installed" as success.
- **Settings-JSON integration**: chosen Consul / Nomad paths written to `settings.json` so the GUI's Service Network tab can launch the agents without further configuration.
- **Legacy install steps deactivated**: Erlang/OTP, RabbitMQ Server, and ProcessHub install flows wrapped in `ENABLE_LEGACY_BROKER` / `ENABLE_LEGACY_PROCESSHUB` conditional-compile guards (default off; flip to 1 to re-enable).
- **Linux**: new `bootstrap.sh` interactive script shipped in `extraResources` for AppImage users, plus a `.deb` postinst hook that launches it when stdin is a TTY. Detects distro, installs from HashiCorp's official apt / yum repos with version pinning, falls back to release-tarball download.
- **Installer size**: cut from ~259 MB to ~30–40 MB compressed by excluding `examples/_legacy/` (~393 MB unpacked of pre-migration RabbitMQ-era templates), trimming `dist-msys2/` and `deploy/` build artefacts, and restricting Chromium locales to `en-US`.

Robot Framework test path: QConnectBase `GrpcClient`

- New `QConnectBase/grpc/grpc_client.py` contributed upstream to QConnectBase. Subclasses `ConnectionBase` and integrates with the standard `Connect` / `Send Command` / `Verify` / `Disconnect` keyword library.
- Two addressing modes: direct (`target: host:port`) and Consul-resolved (`service_name` + `consul_addr`, lazy import of `MicroserviceBase.runtime.consul.resolve_via_consul`).
- `wait_4_trace` overridden to ignore the regex `search_obj` (gRPC returns structured objects, not text streams) and to return the response as a Python `dict`. Tests assert content with Robot's built-in `Should Be Equal` / `Dictionary Should Contain`.
- Reflection client (`GrpcReflectClient`) with automatic fallback to `LocalProtoClient` when the server lacks `grpc++_reflection` (e.g. C++ services built against the vcpkg grpc port).

C++ runtime as standalone CMake / vcpkg package

- `runtime_cpp/CMakeLists.txt` now exports `MicroserviceBaseConfig.cmake` + `MicroserviceBaseTargets` via `install(TARGETS ... EXPORT)`. After `cmake --install`, generated services can `find_package(Microser CONFIG REQUIRED)` from anywhere.
- New vcpkg overlay-port at `ports/microservice-base/`.
- Generator emits a hybrid CMake block: tries `find_package` first, falls back to `add_subdirectory(.../runtime_cpp` when the path resolves (i.e. the service lives inside `<framework>/examples/`).
- Three target names exported for back-compat: `microservice_base::runtime` (modern alias), `microservice_base::microservice_base_runtime` (vcpkg style), `microservice_base_runtime` (legacy bare name).

Python monorepo and multi-proto generators

- `python_tmpl.generate_monorepo()` mirrors the C++ monorepo emitter: one project, N service modules sharing a single `.proto`, one `pyproject.toml` registering each service as a `[project.scripts]` entry point.
- Service Creator wizard's monorepo radio is now valid for both Python and C++.

Documentation refresh

- New canonical `docs/diagrams/00_canonical_architecture.puml` (cluster topology) and `component.puml` (per-workload component layout), both adapted from the TA reference architecture.
- Class diagrams (`class_domain`, `class_ports`, `class_adapters`) rewritten to reflect the post-migration framework boundary.

- Sequence diagrams `sequence_communication`, `sequence_rpc`, `sequence_registration`, `sequence_shutdown` redrawn for the gRPC + Consul + Nomad path.
- 14 pre-migration diagrams archived under `docs/diagrams/_archive/` with a mapping README pointing to their successors.
- Service Creator wizard documentation rewritten end-to-end (Markdown + HTML mirror) to match the current 5-step wizard, with new screenshots for Step 2 (server toolchain), the picker modal (3 layout options), Step 5 review (layout badge), and the new GUI Support tab.
- Full audit pass on framework + scaffold + Tier-A source files brought docstrings into the QConnect house style (column-0 content, **Arguments:** / **Returns:** blocks).
- All 24 live PUML diagrams re-rendered with PlantUML 1.2021.01; 16 stale PNGs archived under `docs/imgs/_archive/`.

Dependency change

- `grpcio-tools` promoted from the `[proto-tools]` optional extra into the base `dependencies` list. The Service Creator wizard's *Pre-generate proto stubs* feature needs it at runtime — it was previously a silent no-op when the bridge's Python lacked it. Cost: ~30 MB on every install.
- `[project.optional-dependencies] proto-tools` block removed; `all` extra updated accordingly.

85.1.2 Version 2.0.0 - 09.02.2026

Major restructure: hexagonal architecture, dual-host GUI, multi-broker support, local process hub, standalone installer.

Architecture

- **Hexagonal Architecture**: Reorganized codebase into domain, ports, and adapters layers
- **Factory Pattern**: All components created through `factory.py` for easy swapping
- **Port Interfaces**: `TransportPort`, `RegistryPort`, `UIBridgePort`
- **Adapter Implementations**: RabbitMQ transport, RabbitMQ registry, FastAPI bridge, Local Hub Manager, Service Executor

GUI v2.0

- **Dual Hosting**: Electron desktop app and browser mode via FastAPI bridge
- **Pure Web Frontend**: HTML/CSS/JS with Bootstrap 5 CDN (no Node.js dependencies)
- **Namespace**: `window.MicroserviceManager` (alias `MM`) replaces Node.js `global` and `require()` patterns
- **Service Plugins**: Dynamically loaded per-service UI components in `web/services/`
- **Dark Sidebar Theme**: Custom Bootstrap dark sidebar with accordion navigation
- **Login Modal**: Bootstrap modal replaces separate Electron popup window
- **Toast Notifications**: `MM.showToast()` replaces `notifier.notify()` and `dialog.showMessageBox()`

Multi-Broker Support

- **Connection Map**: `MM.connections` tracks per-broker state
- **Reverse Lookup**: `serviceToBroker` maps service names to broker endpoints
- **Progressive Disclosure**: Broker headers shown only with multiple connections
- **Session Persistence**: Connection state saved via `sessionStorage`

Local Process Hub

- **Hub Manager**: Start, stop, and monitor local service processes
- **Service Executor**: Process execution with `CTRL_BREAK_EVENT` for graceful shutdown on Windows
- **Config Persistence**: `hub_processes.json` with `${python}` and `${config_dir}` placeholders preserved across save operations
- **Hub Dashboard**: Real-time process status display in the GUI
- **REST API**: Full hub management via `/api/local-hub/*` endpoints

Service Management

- **Service Import**: Import microservice packages from folders or zip archives
- **Service Creator**: Interactive wizard for scaffolding new microservices
- **Registry Shutdown**: `__registry_shutdown__` sentinel notifies subscribers when a service unregisters

Electron Packaging

- **Standalone Installer**: NSIS installer for Windows (`build.bat`), Linux packages (`build.sh`)
- **Read-only Resources**: Scripts in `resources/python/`, app in `app.asar`
- **Writable User Data**: Settings, config, services, logs in `%APPDATA%/devatservgui/`
- **First-Run Init**: Template files copied from resources on first launch
- **Bridge Lifecycle**: Detached bridge process managed via PID file

Windows Process Fixes

- `SIGBREAK` handler converted to `KeyboardInterrupt` for graceful Python cleanup
- Pika `start_consuming()` replaced with `process_data_events(time_limit=1)` loop for interruptible consumption
- `subprocess.PIPE` blocking prevented by using `FileHandler` instead of `StreamHandler` for logging in subprocess mode

Documentation

- 12 Architecture Decision Records (ADR-001 through ADR-012)
- 12 PlantUML diagrams (overview, architecture, component, class, sequence, state)
- Updated README with architecture diagrams, quick start guide, and API reference
- Package documentation with Introduction, Description, and History

85.1.3 Version 0.1.0 - 02/2023

Initial version.

- Basic microservice framework with RabbitMQ communication
- Service registration and discovery
- Electron-based GUI for service monitoring

MicroserviceBase.pdf

Created at 25.05.2026 - 15:40:49

by GenPackageDoc v. 0.16.0